

DDD in Gleam

A textbook in ten katas

Bryan Maass

2026-05-26

DDD in Gleam

A textbook in ten katas

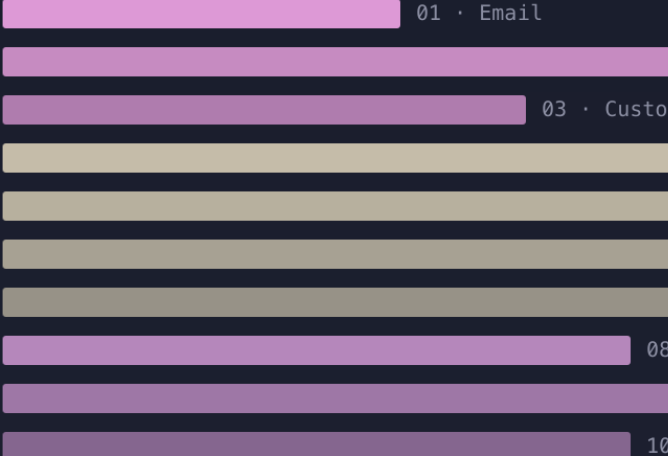
- 
- 01 · Email
 - 02 · Money
 - 03 · Customer
 - 04 · Order
 - 05 · Events
 - 06 · Repositories
 - 07 · Bounded Contexts
 - 08 · HTTP boundary
 - 09 · SQLite repo
 - 10 · Wiring

Table of contents

Welcome	1
Prerequisites	1
How to work through it	1
A note on style	1
00: Introduction	3
What DDD is	3
Why Gleam	3
Why this kata progression	4
How to read this	5
Gleam resources	5
01. Gleam fundamentals for Kata 1	7
Sum types (a.k.a. tagged unions)	7
Records (variants with named fields)	8
opaque: one keyword, one consequence	8
Result(a, e): failure as a value	8
Smart constructors	9
case: Gleam's only conditional	9
Pattern matching on shapes	10
Modules and imports	10
The toolkit so far	11
02. Kata 1: Value Objects (Email)	13
Concept	13
Task	13
Hints	14
Walk-through	15
Critique	15
Out of scope on purpose	15
Takeaway	16
03. Kata 2: Value Objects with operations (Money)	17
Concept	17
New Gleam fundamentals	17
Record update syntax	17
use <-: guards as functions	17
Task	19
Hints: what to do	20

Table of contents

Walk-through	20
Funneling through <code>new</code>	20
<code>use <- require_same_currency(a, b)</code>	20
<code>zero</code> is total	21
Amounts as <code>Int</code> minor units	21
Why <code>add</code> returns <code>Result</code>	21
Critique	21
Takeaway	22
04. Kata 3: Entities (Customer)	23
Concept	23
New Gleam fundamentals	23
Importing types from other modules	23
One-arg <code>use</code> : <code>result.try</code>	23
Task	24
Hints	25
Walk-through	26
Critique	26
Takeaway	26
05. Kata 4: Aggregates (Order)	27
Concept	27
New Gleam fundamentals	27
Internal types (no <code>pub</code>)	27
Multiple guards in sequence	27
<code>list.try_map</code> and <code>list.try_fold</code>	28
Task	28
Hints: what to do	30
Walk-through	31
Critique	31
Takeaway	32
What's next	32
06. Kata 5: Domain Events	33
Concept	33
New Gleam fundamentals	33
Tuples: <code>#(a, b)</code>	33
<code>result.try</code> chaining (earning its keep)	34
Task	34
Hints: what to do	35
Walk-through	36
Scenario testing: commands as data	36
Takeaway	38
Bonus: commands as a first-class concept	39
What's next	39

07. Kata 6: Repositories	41
Concept	41
What you load is what you trust	41
The use case is the orchestrator	41
New Gleam fundamentals	42
Records of functions	42
Why we need OTP	42
OTP, just enough	43
Wrapping the actor in the interface	44
Error wrapping across layers	44
Task	45
Hints: what to do	46
Walk-through	47
Critique	49
Takeaway	49
What's next	50
08. Kata 7: Bounded Contexts	51
Concept	51
The thesis kata 7 demonstrates	51
New Gleam fundamentals	52
Folders as namespace boundaries	52
Type aliases for function types	52
Task: one file per step	52
1. <code>src/shipping/shipment.gleam</code> : the Shipping aggregate	53
2. <code>src/shipping/shipment_repo.gleam</code> : the Shipping repository	54
3. <code>src/shipping/handle_order_placed.gleam</code> : the cross-context handler	54
4. <code>test/shipping/shipment_test.gleam</code> : Shipping aggregate tests	55
5. <code>test/shipping/handle_order_placed_test.gleam</code> : the integration spec	55
Hints: what to do	55
Walk-through	56
Critique	57
Takeaway	58
09. Kata 8: Composition Root + HTTP Boundary	59
Concept	59
Why this is the FCIS payoff chapter	59
New Gleam fundamentals	60
Wisp: the HTTP framework	60
Mist: the underlying HTTP server	61
Wisp/simulate: testing in-process	61
The Deps closure pattern (DI in chill mode)	61
Task	62
1. <code>src/main.gleam</code> : the composition root	62
2. <code>src/web/router.gleam</code> : request to response dispatcher	63
3. <code>src/web/place_order_handler.gleam</code> : the one HTTP handler	63
4. <code>test/web/place_order_handler_test.gleam</code>	63

Table of contents

Endpoint contract	63
Hints: what to do	64
Walk-through	64
Kicking the tires with Hurl	65
Critique	67
Takeaway	67
10. Kata 9: SQLite Repository	69
Concept	69
Why SQLite for a learning project	69
The interesting design problem	69
New Gleam fundamentals	70
<code>sflight</code> : SQLite from Gleam	70
<code>gleam/json</code> : encoding and decoding JSON	71
The snapshot/restore back door pattern	72
Task	73
1. Edit <code>src/order.gleam</code> : add the snapshot back door	74
2. Create <code>src/order_repo_sqlite.gleam</code> : the SQLite adapter	74
3. Create <code>test/order_repo_sqlite_test.gleam</code>	74
Hints	75
Walk-through	75
Critique	76
Takeaway	77
What's next	77
11. Kata 10: Wiring and Configuration	79
New Tricks	79
<code>envoy</code> handles env var access	79
Organizing the Config	80
Config -> Repo	81
Task	82
1. Add the dependency	82
2. Add <code>RepoBackend</code> and <code>Config</code> types	82
3. Implement <code>load_config</code>	82
4. Implement <code>build_repo</code>	82
5. Update <code>main</code>	82
6. Update <code>.gitignore</code>	83
7. Verify persistence	83
Hints: what to do	84
Walk-through	84
Critique	85
Takeaway	86
What's next	86
12. Putting It Into Practice	87
Don't start with all of it	87
Minimum viable DDD	88

Where bounded contexts come from	88
Refactor moves as the system grows	89
“My CRUD endpoints are getting hard to test”	89
“I keep re-validating the same string in 8 places”	89
“I have a 12-field record and I add one more every sprint”	89
“Every endpoint takes 9 arguments now”	89
“I want X to happen after Y but X shouldn’t know about Y”	89
“Tests are slow because they hit the database”	90
“Two teams are stepping on each other in the same module”	90
“Restarting the server loses everything”	90
Signs you’ve over-applied DDD	90
Testing in a DDD-shaped pyramid	91
Composition root in production	91
Talking to non-DDD colleagues	92
What’s beyond the foundation	93
1. Event sourcing	93
2. CQRS: Command/Query Responsibility Segregation	94
3. Sagas / Process Managers	94
4. Distributed event delivery	94
5. Anti-corruption layers in earnest	94
What the kata omits, and what you’d add to ship it	95
Customer endpoints	95
JSON request bodies	95
A few patterns the book skipped on purpose	95
A few things that aren’t DDD but pair well with it	96
A short closing	96

Appendices 97

use in Gleam — an in-depth explainer	97
1. The mechanical rule	97
Why Gleam has this	97
2. The shape of the helper	98
3. Pattern — guard / short-circuit (zero-arg callback)	99
4. Pattern — Result chaining (one-arg callback)	100
5. Pattern — Option chaining	101
6. Pattern — resource acquisition (with-style)	102
7. Pattern — list iteration (less common)	102
8. Pattern — custom DSLs	103
9. Syntactic variants	103
10. Type constraints (the part that bites)	104
11. When NOT to reach for use	104
12. Mental model summary	105

Welcome

A 10-kata progression that teaches idiomatic Gleam through the vocabulary of Domain-Driven Design. Each chapter pairs one DDD concept with the smallest Gleam toolkit that lets the compiler check it.

The chapter labels say DDD; the lessons are idiomatic Gleam.

Prerequisites

- A working Gleam toolchain (`gleam` on your `PATH`).
- Comfort with one statically-typed functional or object-oriented language. The book assumes no prior Gleam.

How to work through it

1. Read the concept and fundamentals sections.
2. Open the source file under `src/` and try the kata before peeking. The tests under `test/` are the spec.
3. Run `gleam test` to confirm.
4. Read the walk-through and critique, then compare to your version.

The reference solutions live in `src/`; the tests in `test/` are their spec. The companion repository is at github.com/escherize/gleam-katas.

A note on style

Each chapter follows roughly this shape:

1. **Concept:** the DDD idea in plain language.
2. **New Gleam fundamentals:** what this kata needs that earlier ones didn't cover. (Omitted when a chapter introduces no new language features.)
3. **The task:** function signatures and rules.

Welcome

4. **Hints:** enough to unblock without spoiling the design.
5. **Walk-through:** the reference solution, with reasoning.
6. **Critique:** what holds up, what shifts as the system grows.
7. **Takeaway:** the property the code now *guarantees*, and why it matters past the toy example.

00: Introduction

A hands-on walkthrough of Domain-Driven Design fundamentals, using Gleam to make the patterns *enforceable* rather than aspirational. Each chapter pairs one DDD concept with the Gleam features that make it compile-time true.

The book lives where the two subjects meet, and neither one receives a full survey of its own.

What DDD is

Domain-Driven Design organizes software so the business vocabulary and its invariants become the load-bearing structure of the code, not an afterthought layered on top of CRUD models.

A few ideas do most of the work. Illegal states should be unrepresentable; if “an order with zero lines cannot be placed” is a rule, the type should *forbid* it rather than every call site *checking* for it. Concepts have shapes, so a `CustomerId` deserves its own type, and so does `Money`, which makes the rest of the system more honest about what it handles. Behavior lives with the data, so “an order is placed” is a method on the order itself, with its own preconditions, returning either a new state or a typed reason it can’t transition. No service elsewhere flips a flag on its behalf.

Most introductions to DDD bury these ideas under pattern catalogs: Repositories, Factories, Anti-Corruption Layers. Those patterns are real, but they sit downstream of the ideas above.

Why Gleam

Gleam is a compact language with the feel of a careful editor’s pass. It ships with very few escape hatches. `let assert`, `panic`, and FFI exist, but each one is a deliberate keystroke rather than an idiomatic shortcut.

- [Algebraic data types](#) with exhaustive [pattern matching](#). You name every alternative, and the compiler refuses to let you forget one.

- **Opaque types**, a one-keyword feature that lets a module hide its constructor. Outside the module, the type exists, but the only way to build one is through functions you expose.
- Failure is a value rather than an exception or a null (**Result(a, e)**, **Option(a)**). Callers can't pretend it didn't happen.
- Every branch is a **case** rather than an **if**, so you must consider the alternatives by name.
- One control-flow primitive (**use <-**) covers guards, chaining, resource handling, and whatever custom flow you'd otherwise invent.

These features map onto what DDD wants:

DDD wants...	Gleam gives you...
Illegal states unrepresentable	Opaque types + smart constructors
Validate at the boundary, trust within	Result + types as proof
Named, exhaustive failure modes	Sum-type errors + case
Aggregates as consistency boundaries	Modules + opacity + record update syntax
Linear top-to-bottom domain rules	use <- for guard chains

You can do DDD in any language. Gleam fits it well enough that the patterns stop being conventions and start being what the compiler checks.

Why this kata progression

The book climbs from one-file types up to a wired backend, each kata adding one DDD concept and only the Gleam features that concept needs.

1. *Value Objects* (**Email**, **Money**, katas 1–2). Opaque types, smart constructors, error sums, and **use** chaining.
2. *Entities* (**Customer**, kata 3). The same toolkit, but identity matters; ID types, equality semantics, and state transitions modeled as new immutable values.
3. *Aggregates* (**Order**, kata 4). Several internal pieces with invariants spanning them, so the work shifts to hiding internals behind one front door.
4. *Domain Events* (**OrderPlaced** and friends, kata 5). State transitions that *also* report what happened.
5. *Repositories* (kata 6). Records of functions as interfaces; OTP actors hidden behind them.
6. *Bounded Contexts* (kata 7). Folders as boundaries; events as the integration contract.
7. *Composition Root + HTTP Boundary* (kata 8). Wisp/Mist with a **Deps** record. The shell translates; the core decides.

8. *SQLite Repository* (kata 9). Same interface, real persistence, snapshot/restore for opaque aggregates.
9. *Wiring and Configuration* (kata 10). Env vars become typed config; a factory routes config to adapters.

By the end you can read or write idiomatic Gleam domain code without having taken either subject as a course first.

How to read this

Each chapter follows the same shape:

- *Concept*: the DDD idea in plain language.
- *Gleam fundamentals you need*: enough to do the kata, not a full language tour.
- *The task*: a function signature and the rules it must satisfy.
- *Hints / what to do*: nudges to unblock without spoiling the design.
- *A walk-through*: the reference solution, with reasoning.
- *Critique*: where the solution holds up, where it could be tightened, and how it strains as the system grows around it.
- *Takeaway*: what the code now *guarantees*, and why that matters past the toy example.

Type the code and satisfy the tests before you read the walk-through, because the exercises won't do their work otherwise.

Gleam resources

Useful tabs to keep open while you work through the book:

- [The Gleam Language Tour](#), interactive and in-browser, and the best place to look up any language feature.
- [Standard library reference](#): `gleam/string`, `gleam/list`, `gleam/result`, and the rest. You'll reach for these in every kata.
- [Gleam documentation index](#), with guides and cheatsheets, including a [Gleam-for-Rust](#) and [Gleam-for-Elixir](#) one.
- [Gleam home](#) for install instructions if you don't already have the `gleam` CLI.

01. Gleam fundamentals for Kata 1

Just enough Gleam to do Kata 1; each chapter adds to the toolkit. Readers fluent in Gleam can skim the headers.

Sum types (a.k.a. tagged unions)

Declare a custom type as a fixed set of *variants*:

```
pub type Currency {  
  USD  
  EUR  
  GBP  
}
```

A `Currency` value is exactly one of `USD`, `EUR`, or `GBP`. There's no `null` and no string `"USD"` impersonating the variant; the type has three values and that's the set.

Variants can carry data:

```
pub type Shape {  
  Circle(radius: Float)  
  Rectangle(width: Float, height: Float)  
}
```

`Circle(3.0)` and `Rectangle(2.0, 5.0)` are both `Shape` values, the same type holding differently shaped data, hence “sum type.”

Records (variants with named fields)

A type with a single variant of the same name is Gleam's record/struct:

```
pub type Email {  
  Email(value: String)  
}
```

Construction is `Email("foo@bar.com")` and field access is `email.value`. The type and the variant sharing a name is idiomatic Gleam.

opaque: one keyword, one consequence

A custom type's constructor is public by default and travels with the type; `opaque` keeps the constructor private to its module:

```
pub opaque type Email {  
  Email(value: String)  
}
```

Outside this module the *type* `Email` still exists and behaves like any other value, but the constructor `Email(...)` is gone and field access `email.value` no longer compiles. The only way to get an `Email` is to call a function the module exports, which validates whatever it wants before wrapping the value, so the type system guarantees nothing else gets through.

That mechanism is the foundation for “make illegal states unrepresentable,” and forgetting `opaque` is the most common mistake when starting out.

`Result(a, e)`: failure as a value

Gleam has no exceptions, so a function that can fail returns a `Result`:

```
// Built-in. Conceptually:  
pub type Result(a, e) {  
  Ok(a)  
  Error(e)  
}
```

```
}

```

A typical signature is `Result(ValidThing, SomeError)`, where `Ok(v)` holds the success value and `Error(e)` holds why it failed.

Callers **case** on the result to reach either branch, and there's no way to grab the value without acknowledging failure. (`let assert Ok(x) = ...` exists for crash-on-error scenarios, mostly tests.)

Smart constructors

A *smart constructor* is the function outside callers use to build an opaque value. It validates and either wraps the value in the type or returns a typed error:

```
pub fn new(raw: String) -> Result(Email, EmailError) {
  // ...validate, then either Ok(Email(...)) or Error(...)
}
```

Convention names it **new**; combined with **opaque**, it becomes the only door into the type.

case: Gleam's only conditional

Gleam has no **if**, only **case**, which pattern-matches on a value:

```
case x {
  0 -> "zero"
  1 -> "one"
  _ -> "other"
}
```

The compiler checks *exhaustiveness*: a missed variant in a match on a sum type breaks the build. “Named failure modes” stop being a discipline and become a compiler-enforced property.

`_` matches anything and serves as the catch-all once the rest are explicit.

Pattern matching on shapes

`case` can destructure as it matches, so patterns end up describing shapes:

```
case string.split(input, "@") {
  [""]      -> Error(Empty)           // single empty element
  [_]       -> Error(MissingAt)       // single non-empty element
  ["", _]   -> Error(MissingLocal)    // empty before @
  [_, ""]   -> Error(MissingDomain)   // empty after @
  [_, _]    -> Ok(...)                // exactly two non-empty parts
  -         -> Error(TooManyAt)       // three or more
}
```

Each branch names a shape the input could take, so the structure of the data does the work that chained guards do elsewhere. The surprise coming from imperative languages is that a sequence of guards like `if (!hasAt) ... else if (!domain) ...` collapses into one `case`.

Order of patterns matters when they overlap: `[""]` must come before `[_]` because the empty string matches both, and `case` picks the first.

Modules and imports

One file is one module, named after the file, and public items use `pub`:

```
import gleam/string // standard library
import email        // sibling module in the same project

pub fn foo() { string.trim(" hi ") }
```

A module's contents are reachable as `module.name`, or specific names can come into scope:

```
import email.{type Email} // brings the type into scope unqualified

pub fn handle(e: Email) { ... }
```

Functions stay namespaced as `email.new(...)`; only the type alias is unqualified.

The toolkit so far

- A type defined with one or many variants.
- `opaque` to restrict construction to the defining module.
- A smart constructor returning `Result(YourType, YourError)`.
- Validation via `case` on the structure of the input.
- Either `Ok(YourType(...))` or `Error(NamedReason)` as the return.

The rest of the book waits on an attempt at the kata.

02. Kata 1: Value Objects (Email)

Concept

A **value object** has no identity beyond its attributes. Two **Email**s with the same string are the same email, the way 5 and 5 are the same 5. Nothing distinguishes one from another beyond the string itself.

The exercise enforces one rule: you cannot construct an invalid **Email**. Validate once at the boundary where the string arrives, and from then on any function taking an **Email** parameter can trust it without defensive checks.

In Gleam that means an opaque type with a smart constructor returning **Result**.

Task

Create `src/email.gleam` exposing:

```
pub opaque type Email {
  Email(value: String)
}

pub type EmailError {
  Empty
  MissingAt
  // add more as you see fit
}

pub fn new(raw: String) -> Result(Email, EmailError) {
  // your code
}
```

02. Kata 1: Value Objects (*Email*)

```
pub fn to_string(email: Email) -> String {  
    // your code  
}
```

Rules:

- Trim leading/trailing whitespace.
- Reject empty (after trimming).
- Exactly one @.
- Both sides of @ non-empty.

The tests in `test/email_test.gleam` are the spec. Run with `gleam test`.

Hints

1. Trim first. `gleam/string` has `string.trim/1`, and trimming before any structural check folds a class of edge cases into the empty case.
2. Skip boolean guards. Don't write `if has_at && local_is_non_empty && domain_is_non_empty`. Gleam has no `if`, so a cleaner shape opens up.
3. Inspect what `string.split(trimmed, "@")` returns. It's a `List(String)`, and the shape of that list (its length, which elements are empty) *is* the validation. Sketch every meaningful shape on paper before writing code.
4. Pattern order matters. When two patterns can match the same input (e.g. `[]` and `[_]`), put the more specific one first. The empty string matches both, but the right name for it is `Empty`, not `MissingAt`.
5. Add error variants as you discover them. The starter has `Empty` and `MissingAt`. At least two more meaningful failure modes are waiting. Name each one, because that's the point of a sum-type error.
6. For `to_string`, reach into the inner field directly. Inside the module that works because the constructor is in scope; outside, it wouldn't.

If 15 or 20 minutes pass without traction, scroll down. Brute-forcing it defeats the point; the goal is to internalize the pattern.

Walk-through

`opaque` does the work, since nothing outside this module can build an `Email` without calling `new`. Value objects need exactly that guarantee.

Check `[]` before `[_]`: an empty string after trimming splits into `[]` (a one-element list holding the empty string), which also matches `[_]`. When patterns overlap, the more specific one goes first.

A typed error beats a `Bool` because the variant names which failure happened, and the compiler forces every caller to handle both branches. That information survives all the way out to wherever the error surfaces, whether that's an HTTP response or a log line.

Use `Ok(Email(trimmed))` rather than `Ok(Email(local <> "@" <> domain))`. Some solutions rebuild the email from the split parts, but the split already did its job as validation and `trimmed` is the string you want.

Critique

- After splitting on `@`, `[_]` matches a non-empty token, so the input had no `@`. `MissingAt` reads correctly here.
- `[]` only happens when `trimmed` is `""`. Trimming first folds whitespace-only inputs into the `Empty` case for free.
- A name like `MissingTextBeforeAt` reads better than `EmptyLocalPart` for the audience that sees these errors. Domain language is part of the design.

Out of scope on purpose

This validator is a teaching exercise, not a production email parser. What it doesn't do, and what you'd add if it had to ship:

- **RFC 5322 corner cases.** Quoted local parts, comments, escaped `@` characters, IP-literal domains, all rejected here and all legal in the RFC.
- **Internationalized email (RFC 6531).** `user@bücher.de` and `@example.com` both fail the “ASCII-friendly” assumptions baked into the split. A real validator either pre-encodes to IDNA/punycode or accepts Unicode locals.
- **Whitespace inside the address.** Trimming the outside isn't the same as rejecting `"@example.com"`, which currently passes both halves of the `[_]` check with a leading space.

02. Kata 1: Value Objects (*Email*)

- **Deliverability.** Even a syntactically perfect address may not exist. That's an MX-lookup / send-and-confirm problem, not a parser problem.

The point of the kata is that *whatever* rules you settle on, the opaque-type-plus-smart-constructor shape makes them enforceable in one place. Swap in a stricter parser tomorrow and the rest of the codebase doesn't move.

Takeaway

Anywhere in your codebase with a parameter `email: Email`, the compiler guarantees `new` already validated it. You validate once at the boundary and the type carries the proof everywhere else, so downstream code never has to recheck.

That's the upfront ceremony the design pays for. Every later layer of the system assumes the email is well-formed, because the type system permits nothing else.

03. Kata 2: Value Objects with operations (Money)

Concept

Same shape as `Email` (opaque type, smart constructor), under two new pressures.

1. A `Money` has two attributes (amount and currency) that have to stay coherent.
2. `add(usd, eur)` is nonsense the type system should refuse.

The rules of the domain become rules of the type. The type carries the operations that respect its invariants alongside the data they operate on.

New Gleam fundamentals

Kata 1 needed an opaque type and a smart constructor. This one adds record update syntax and the `use <-` desugaring.

Record update syntax

To produce a modified copy of a record without re-listing every field:

```
let updated = Money(..money, amount: 0)
```

`..money` says “take all the fields from this value, then override the named ones.” Without it, aggregate code drowns in field repetition.

`use <-`: guards as functions

The same precondition shows up at the head of every operation:

03. Kata 2: Value Objects with operations (Money)

```
case a.currency == b.currency {  
  False -> Error(CurrencyMismatch)  
  True -> {  
    // do the work  
  }  
}
```

Pull the precondition into a helper that takes a *callback*:

```
fn require_same_currency(  
  a: Money,  
  b: Money,  
  then: fn() -> Result(t, MoneyError),  
) -> Result(t, MoneyError) {  
  case a.currency == b.currency {  
    False -> Error(CurrencyMismatch)  
    True -> then()  
  }  
}
```

Then call it with `use`:

```
pub fn add(a: Money, b: Money) -> Result(Money, MoneyError) {  
  use <- require_same_currency(a, b)  
  new(a.amount + b.amount, a.currency)  
}
```

`use <- f(...)` is sugar for `f(..., fn() { rest_of_block })`. It lifts the remainder of the block into a callback and passes it as the helper's last argument.

A few mechanical facts:

- `use <-` with no binding gives the callback zero arguments, which makes it a guard.
- `use x <- f(...)` gives the callback one argument, for unwrapping (Kata 3).
- The helper decides whether to call the callback. On a currency mismatch it returns its own error and the rest of the block never runs.
- The lowercase `t` in `Result(t, MoneyError)` is a *generic* type variable. The helper works whether the calling body returns `Result(Money, ...)` or `Result(Order, ...)`.

Aggregates built on this pattern read top-to-bottom like a list of business rules. For a deeper explainer, see [use.md](#).

Task

Create `src/money.gleam` exposing:

```
pub type Currency {
  USD
  EUR
  GBP
}

pub opaque type Money {
  Money(amount: Int, currency: Currency)
}

pub type MoneyError {
  NegativeAmount
  CurrencyMismatch
}

// Amount is in minor units (cents/pence). $1.50 -> new(150, USD).
pub fn new(amount: Int, currency: Currency) -> Result(Money, MoneyError)
pub fn add(a: Money, b: Money) -> Result(Money, MoneyError)
pub fn subtract(a: Money, b: Money) -> Result(Money, MoneyError)
pub fn multiply(money: Money, factor: Int) -> Result(Money, MoneyError)
pub fn same_currency(a: Money, b: Money) -> Bool
pub fn zero(money: Money) -> Money
```

Rules:

- Reject negative amounts in `new`.
- `add` and `subtract` fail with `CurrencyMismatch` if currencies differ.
- `subtract` shouldn't produce negative money (returns `NegativeAmount`).
- `multiply` with a negative factor should also be rejected.
- `zero(money)` returns a money of the same currency with amount 0.

The tests in `test/money_test.gleam` are the spec.

Hints: what to do

1. Write the naive version first, with each operation doing its own checks: `subtract` checks currencies and verifies the result isn't negative, `multiply` checks negativity. Get the tests passing before refactoring.
 2. Then look for duplication. Operations that touch two `Money`s repeat the same currency check, and operations that produce a new `Money` re-check the negative-amount rule on their own.
 3. Route everything through `new` so it becomes the source of truth for the negative-amount invariant. `subtract` loses its own check, and `multiply` inherits the check for free.
 4. Lift the currency check into a helper. Write `require_same_currency` with the shape from the fundamentals section, then call it with `use <-`. Every operation now reads as a guard followed by the computation.
 5. `zero(money)` returns `Money`, not `Result(Money, _)`. The value 0 always satisfies the invariant, so the function is total. Leave `Result` out of the return type when nothing can go wrong.
 6. Trace the test for `multiply(m, -1)`. Why `NegativeAmount`? If `multiply` routes through `new`, the answer falls out, which is the point of funneling.
-

Walk-through

Funneling through `new`

Every constructor path re-validates. `subtract` drops its own `amount >= 0` check because `new` already runs it, and `multiply` catches negative factors for the same reason. The non-negative invariant lives in one place, so no caller can construct a `Money` that violates it.

```
use <- require_same_currency(a, b)
```

A naive solution duplicates the currency check across every operation. Pull the check into a named helper and each operation turns into a flat list of business rules, where every entry is “require same currency, then do the work.”

The mechanical desugar:

```
pub fn add(a: Money, b: Money) -> Result(Money, MoneyError) {
  require_same_currency(a, b, fn() {
    new(a.amount + b.amount, a.currency)
  })
}
```

`use` is sugar for that callback shape, so reading one form teaches the other.

zero is total

The function returns `Money` rather than `Result(Money, _)` because `0` always satisfies the non-negative rule. Record update (`Money(..money, amount: 0)`) carries the currency through.

Amounts as Int minor units

Floats and money don't mix, because $0.1 + 0.2 \neq 0.3$ in IEEE-754, which is why banks store cents instead of dollars. Choosing `Int` here is a domain modeling decision the type records.

Why add returns Result

A primitive `add(usd_5, eur_3)` silently hands back `8` of nothing. The domain-modeled version returns `CurrencyMismatch`, so the bug now sits in the type signature where the caller has to deal with it.

Critique

- `same_currency` is `pub` as a forward lean: the next kata (`Order`) wants to compare line currencies without reaching for `==` on the currency field, and a named predicate reads better than `a.currency == b.currency` at the call site. Strictly speaking, keeping it private and inlining one comparison in `Order` would be the smaller diff. Pick whichever rule you prefer (“expose only what’s used” vs “expose the named predicate when one exists”) and stay consistent. The book picks the latter. The private helper (`require_same_currency`) stays private because it’s a `use <-` callback shape, not a predicate.
 - The module ignores integer overflow, which is fine for toy domain code. For production money handling, use a `bigint` type or guard explicitly.
-

Takeaway

The module has one way to produce a **Money**, and that path passes every invariant. No caller can lie about currency or smuggle in a negative amount, so every **Money** in the system is valid by construction.

“Make illegal states unrepresentable” is the slogan, and the type now embodies it.

04. Kata 3: Entities (Customer)

Concept

A customer named “Alice” who renames to “Alice Smith” tomorrow remains the same customer, because identity persists while attributes change.

Equality runs on the ID. Gleam’s `==` lies about entities; two snapshots of one customer at different moments compare unequal because their fields differ, though the person is one. Entities want an explicit ID type and a `same_X` function that compares by ID, so `==` stays out of entity code.

Value objects start paying rent here. `Customer` *contains* the `Email` from Kata 1, so the customer layer skips re-validation; the type already carries the proof.

New Gleam fundamentals

Importing types from other modules

```
import email.{type Email}
```

This pulls the *type* `Email` into scope by bare name, so the annotation reads `email: Email` rather than `email: email.Email`. Functions still need their namespace (`email.new(...)`), since only the type alias travels unqualified.

One-arg use: `result.try`

`Customer` composes a `CustomerId` and an `Email`, both of which run through smart constructors, so a boundary function that turns raw strings into a `Customer` chains three constructions that can each fail:

```
case email.new(raw_email) {  
  Error(e) -> Error(InvalidEmail(e))  
}
```

04. Kata 3: Entities (Customer)

```
Ok(email) -> case new_id(raw_id) {  
  Error(e) -> Error(e)  
  Ok(id) -> new(id, raw_name, email)  
}  
}
```

use flattens that pyramid:

```
use email <- result.try(email.new(raw) |> result.map_error(InvalidEmail))  
use id <- result.try(new_id(raw_id))  
new(id, raw_name, email)
```

`result.try(r, fn(value) { ... })` runs the callback only when `r` is `Ok`; on `Error` the expression short-circuits to that error, so `use` turns the call site into a readable sequence of validations. `result.map_error(f)` rewraps an inner error to fit the outer error type when the two don't line up.

The minimal Kata 3 won't force you onto this, but you'll reach for it the moment you start composing types. See `use.md` for the deeper treatment.

Task

Create `src/customer.gleam` exposing:

```
import email.{type Email}  
  
pub opaque type CustomerId {  
  CustomerId(value: String)  
}  
  
pub opaque type Customer {  
  Customer(id: CustomerId, name: String, email: Email)  
}  
  
pub type CustomerError {  
  EmptyName  
  EmptyId  
}
```

```

pub fn new_id(raw: String) -> Result(CustomerId, CustomerError)
pub fn new(id: CustomerId, name: String, email: Email) -> Result(Customer, CustomerError)
pub fn id(customer: Customer) -> CustomerId
pub fn rename(customer: Customer, new_name: String) -> Result(Customer, CustomerError)
pub fn change_email(customer: Customer, new_email: Email) -> Customer
pub fn same_customer(a: Customer, b: Customer) -> Bool

```

Rules:

- `CustomerId` is its own opaque type; never use raw strings as IDs in domain code.
- Reject empty names and empty ID strings.
- `rename` and `change_email` return a *new* `Customer` with the same ID.
- `same_customer` compares by ID only.

The tests in `test/customer_test.gleam` are the spec. The file ships empty, so cases land as the implementation lands.

Hints

1. Treat this as Kata 1 applied twice. `CustomerId` is an opaque value object with its own smart constructor, `new_id`, and `Customer` is an opaque entity whose `new` takes value objects that already carry their proofs.
2. `change_email` returns `Customer` rather than `Result` because its one substantive argument, `new_email`, already carries `Email`'s proof, so nothing remains to fail on. The value-object layer pays out.
3. `rename` does return `Result`, because `new_name: String` is raw input that can still be empty. Route it through `new` so the empty-name check lives in one place.
4. For `same_customer`, plain `==` on `CustomerId` is the right tool. `CustomerId` is opaque and only constructable through `new_id`, so every `CustomerId` in your program already cleared the constructor, and opacity buys you trustworthy equality.
5. Skip the extra fields (`created_at`, `version`, address, whatever else is tempting). The kata wants the entity-vs-value distinction; everything else is noise.
6. The strict `new` takes values that already carry their proofs, so don't let raw strings through. Raw input belongs in a separate boundary factory that pipelines the three smart constructors together; see `use.md`. Splitting the two keeps each function honest about what it can fail on.

Walk-through

Funnel `rename` through `new`, same instinct as `Money`. The function writes no validation of its own and hands off to the smart constructor, which already checks the name, so the invariant lives in one place.

`Email` arrives carrying its proof, so the entity layer skips the string check; the type already establishes the invariant. Compounding like this earns value objects their keystrokes.

`same_customer` carries the other half. After a rename, `alice_v1 == alice_v2` returns `False` because the structs differ, yet domain-wise they name one person, and a named function makes the caller commit to identity equality instead of stumbling into structural equality by reflex. `CustomerId` stays opaque and only `new_id` builds one, so `a.id == b.id` is safe; both IDs went through the constructor.

Critique

- `string.length(raw) == 0` works, but `string.is_empty(raw)` is cheaper (no traversal) and reads as the intent. `new_id` above uses `is_empty` while `new` still uses `length`; pick one.
- `new(... name: " ")` succeeds as written, because whitespace-only names pass the check. Whether that counts as a bug depends on the domain, and every spot a string enters your domain is a place for invariants to leak.
- The strict `new` is good design that grates when you genuinely have raw input from an HTTP handler. Add a separate `from_raw` boundary factory that chains the three smart constructors in a `use-chain`; `use.md §4` has the worked example.

Takeaway

You have a `Customer` whose attributes drift while its identity stays fixed and trusted. `rename` and `change_email` are state transitions. Each produces a new immutable value, the next moment in that customer's life.

An event-sourced system would emit a `CustomerRenamed` or `EmailChanged` event on each transition, and replaying them would reconstruct the customer's history. Your design already fits that mold without trying; Kata 5 turns the theory into working code.

05. Kata 4: Aggregates (Order)

Concept

An aggregate clusters related objects into a unit. The aggregate root is the only entry point; everything outside goes through it. The root enforces invariants that span multiple internal pieces.

An `Order` has `OrderLines`. Rules like “an order must have at least one line to be placed” or “every line must share a currency” are properties of the aggregate, which no individual line can enforce on its own, so `OrderLine` stays internal, and `Order` is the only thing the outside world touches.

Gleam’s module system starts pulling its weight here. A module is the natural unit for an aggregate, and the module seals anything not marked `pub`.

New Gleam fundamentals

Internal types (no `pub`)

A type without `pub` is private to its module:

```
type OrderLine {  
  OrderLine(sku: String, quantity: Int, unit_price: Money)  
}
```

Outside the module, `OrderLine` does not exist as a name. `Order` exposes operations that manipulate lines internally, and nothing outside can construct or hold one. The type system enforces the rule against reaching into the aggregate.

Multiple guards in sequence

`use` <- carried one guard in Kata 2. Aggregate invariants stack several:

05. Kata 4: Aggregates (*Order*)

```
pub fn add_line(order, sku, quantity, unit_price) {  
  use <- no_modify_placed(order)  
  use <- non_empty_sku(sku)  
  use <- positive_qty(quantity)  
  use <- currency_matches(order.lines, unit_price)  
  // ... do the work  
}
```

Each line names one precondition, and the body reads top-to-bottom like a spec.

`list.try_map` and `list.try_fold`

For aggregating over a list when each step can fail:

- `list.try_map(list, f)` runs `f` on each element and short-circuits on the first **Error**. Returns `Result(List(b), e)`.
- `list.try_fold(list, init, f)` folds with a fallible combining function and short-circuits the same way.

`order.total` uses both: multiply each line's price by its quantity, then sum the results. Either stage can fail.

Task

Create `src/order.gleam` exposing:

```
import customer.{type CustomerId}  
import money.{type Money}  
  
pub opaque type OrderId {  
  OrderId(value: String)  
}  
  
// Internal. Outside this module, this type does not exist.  
type OrderLine {  
  OrderLine(sku: String, quantity: Int, unit_price: Money)  
}
```

```

pub type OrderStatus {
    Draft
    Placed
}

pub opaque type Order {
    Order(
        id: OrderId,
        customer_id: CustomerId,
        lines: List(OrderLine),
        status: OrderStatus,
    )
}

pub type OrderError {
    EmptyOrderId
    EmptySku
    NonPositiveQuantity
    CannotModifyPlacedOrder
    CannotPlaceEmptyOrder
    CurrencyMismatch
    InvalidOrderTotal
}

pub fn new_id(raw: String) -> Result(OrderId, OrderError)
pub fn new(id: OrderId, customer_id: CustomerId) -> Order
pub fn add_line(order: Order, sku: String, quantity: Int, unit_price: Money) -> Result(Order, OrderError)
pub fn place(order: Order) -> Result(Order, OrderError)
pub fn total(order: Order) -> Result(Money, OrderError)

```

Aggregate invariants, the reason the type exists:

1. `add_line` rejects empty SKUs and non-positive quantities.
2. `add_line` fails with `CannotModifyPlacedOrder` if status is `Placed`.
3. `add_line` fails with `CurrencyMismatch` when the new line's currency differs from existing lines.
4. `place` fails with `CannotPlaceEmptyOrder` if there are no lines.
5. `place` transitions `Draft` → `Placed`.
6. `place` fails with `CannotModifyPlacedOrder` on an already-placed order.
7. `total` sums all line totals, and fails on an empty order or any underlying money error.

05. Kata 4: Aggregates (*Order*)

The tests in `test/order_test.gleam` are the spec.

Hints: what to do

1. Start with the naive version. `add_line` is one big nested `case`. Get it green, then carve out the helpers. The duplication is the signal.
2. Each invariant becomes a guard helper: `no_modify_placed`, `non_empty_sku`, `positive_qty`, `currency_matches`. Each takes a callback (`then: fn() -> Result(...)`) and returns `Error(SpecificReason)` when the rule is violated, otherwise delegating to the callback.
3. Apply them via `use <-`, stacked in `add_line`, so the body becomes “run the rules, then do the one record update.”
4. State checks come before arg validation. A caller passing `("", -3, ...)` against a placed order sees `CannotModifyPlacedOrder` first, since `Placed` already locks the order out of further modification.
5. Reuse `no_modify_placed` in `place`. “Re-place a placed order” and “modify a placed order” violate the same invariant, so the helper names a domain rule that applies wherever it fits.
6. `Order(..order, lines: [new_line, ..order.lines])` is record update plus list cons. Without record update, changing one field forces retyping every field of `Order`; with it, only the change shows.
7. `total` works in two stages:
 - line-by-line, multiply `unit_price * quantity` with `list.try_map` (each multiply can fail);
 - sum the per-line totals with `list.try_fold`, starting from `money.zero(first.unit_price)` (each `money.add` can fail). Both stages return `Result(_, money.MoneyError)`; translation to `OrderError` uses `result.map_error`.
8. `new` does not return `Result`. A brand-new draft order with no lines is always valid, so `place` enforces “an empty order cannot be placed.” Constructors stay honest about what they validate.

If `add_line` runs longer than ~10 lines after the refactor, something has gone wrong.

Walk-through

`add_line` resolves into a short stack of preconditions with one line of real work underneath. Each `use <-` names a domain rule, and the mutation is a record update at the bottom. Aggregate code reads like a flat list of invariants followed by the transition.

Order of preconditions matters. State checks (`no_modify_placed`) come before arg validation (`non_empty_sku`, `positive_qty`), so a caller passing `("", -3, ...)` against a placed order deserves `CannotModifyPlacedOrder` ahead of `EmptySku`. `Placed` locks out further modification, and the rest is moot.

`place` reuses `no_modify_placed` because “re-place a placed order” and “modify a placed order” violate the same invariant. The helper names a domain rule that applies wherever it fits.

`Order(..order, lines: [new_line, ..order.lines])` is record update plus list cons. Without record update, changing one field forces relisting every field of `Order`; with it, only the change shows.

`total` uses `try_map` then `try_fold`: map each line to its line total with `money.multiply`, then fold those with `money.add`. Both stages short-circuit on the first failure.

The `case order.lines` wrapper around `total` covers two cases:

- A non-empty order matches `[first, ..]`, which supplies a valid currency to seed `money.zero`.
- An empty order has no meaningful total, so `Error(InvalidOrderTotal)` is the honest answer.

Critique

Collapsing `money.MoneyError` to `InvalidOrderTotal` flattens the diagnostic, so an underlying overflow and a currency mismatch from the money layer look identical to the caller. Two options:

- (a) define richer `OrderError` variants (`OrderCurrencyMismatch`, `OrderOverflow`) and translate explicitly;
- (b) wrap the underlying error as `TotalCalculationFailed(money.MoneyError)`.

The book recommends (b). It preserves the cause without exploding the error vocabulary, and it keeps the layering honest: `OrderError` doesn’t have to know what failure modes `Money` will grow next month. Option (a) is the right choice only when the outer layer needs to *react* differently per cause (different HTTP status codes, different retry policies); when callers just want to log the cause, wrapping is cheaper and ages better.

05. Kata 4: Aggregates (*Order*)

`OrderLine` is fully internal, with no `pub` and no `opaque`. The original spec said `pub opaque type OrderLine`, which also provides opacity. Private-only is cleaner because it signals that nothing outside should even name the type. Whether external code ever needs to talk about an `OrderLine` (say, as a return type) decides between the two; in this aggregate, it does not.

`new_id` does not trim, so `new_id(" ")` succeeds with a whitespace ID. That is a bug; the fix mirrors `customer.new_id`.

Takeaway

The `Order` module is now a boundary. Outside it, `OrderLine` is not constructible, and no caller can put an order into an inconsistent state. “Currency homogeneity across lines” stops being a concept anyone has to remember, because the only door in enforces it.

When DDD writers say “the aggregate is a consistency boundary,” they mean the module. Every operation either succeeds with a fully-valid new state or fails with a typed error explaining why.

What's next

Kata 5 takes on domain events: state transitions that also report what happened. `add_line` and `place` will return the new `Order` along with a list of facts (`LineAdded`, `OrderPlaced`) that other parts of the system can react to without the aggregate knowing about them.

The existing toolkit carries straight through: opacity, smart constructors, `use <-` chains, record update. Events add to the aggregate’s return type; nothing needs redesigning.

06. Kata 5: Domain Events

Concept

Until now, every operation on the aggregate returned the new state. From here on, each one also returns a list of facts about what just happened. Those facts, domain events, are immutable past-tense records carrying enough data to describe the transition on their own.

What you get for the extra return value:

- Other parts of the system can react without the aggregate knowing they exist. Send a confirmation email when `OrderPlaced` fires; `Order` doesn't need to know email exists.
- The event stream *is* the audit log, with no separate “what happened to this order” code path.
- Once the aggregate emits a complete event log per state change, you can derive state by replaying events instead of storing it. That's the foundation CQRS and event sourcing build on.

`OrderPlaced` names a fact in the past tense, where `PlaceOrder` would name a command in the present, a request rather than a record. Mix the two and you lose the ability to reason about your system. Commands and events are duals, a point the bonus section returns to.

New Gleam fundamentals

The kata mostly widens the return shape, so the toolkit is what you already know from earlier chapters.

Tuples: `#(a, b)`

Gleam's lightweight pair / triple syntax:

```
let pair = #(order, [OrderCreated(id, cid)])
let #(o, events) = pair
```

06. Kata 5: Domain Events

The pair bundles new state with the events from this transition without a named record per operation.

Pattern matching on tuples works the same as anywhere else:

```
case order.add_line(o, sku, q, p) {  
  Ok(#(new_order, events)) -> todo  
  Error(reason)             -> todo  
}
```

result.try chaining (earning its keep)

You met `result.try` in Kata 3, and `place` is where it earns its keep. The `OrderPlaced` event carries the total, computing that total can fail, and `place` has to thread the preconditions, the total, and the event into one chain.

```
pub fn place(order: Order) -> Result(#(Order, List(OrderEvent)), OrderError) {  
  use <- no_modify_placed(order)  
  case order.lines {  
    [] -> Error(CannotPlaceEmptyOrder)  
    _ -> {  
      use t <- result.try(total(order))  
      let placed = Order(..order, status: Placed)  
      Ok(#(placed, [OrderPlaced(order.id, t)]))  
    }  
  }  
}
```

That `use t <- result.try(total(order))` line is what `use` was designed for. If `total` returns `Error`, the chain stops there; otherwise `t` binds and execution continues. Without it you'd nest a `case` inside another `case`.

Task

Modify `src/order.gleam` so `new`, `add_line`, and `place` return both the updated aggregate **and** the events emitted:

```

pub type OrderEvent {
  OrderCreated(order_id: OrderId, customer_id: CustomerId)
  LineAdded(order_id: OrderId, sku: String, quantity: Int, unit_price: Money)
  OrderPlaced(order_id: OrderId, total: Money)
}

pub fn new(id: OrderId, customer_id: CustomerId) -> #(Order, List(OrderEvent))

pub fn add_line(
  order: Order,
  sku: String,
  quantity: Int,
  unit_price: Money,
) -> Result(#(Order, List(OrderEvent)), OrderError)

pub fn place(order: Order) -> Result(#(Order, List(OrderEvent)), OrderError)

```

Rules:

- `new` emits `[OrderCreated(...)]` and is total (no `Result` wrapper).
- `add_line` emits `[LineAdded(...)]` on success.
- `place` emits `[OrderPlaced(...)]` on success, with the total computed at placement time.
- A failed operation isn't a fact, so the `Error(...)` branch returns the error by itself and never builds a partial event list.

`test/order_test.gleam` holds the per-operation spec, and `test/order_scenarios_test.gleam` exercises the aggregate through sequences of commands (see “Scenario testing” below).

Hints: what to do

1. Implement in this order: `new_id` → `new` → guard helpers → `add_line` → `total` → `place`. `place` depends on `total`, so `total` comes first.
2. `new` is now the simplest function in the file. Build the order, attach the `OrderCreated` event, and return the tuple; no `Result` wrapper and no precondition guards.
3. Event construction lives on the `Ok` branch only. When a precondition guard returns `Error(...)`, the `use <-` chain short-circuits and you never reach the line that builds the event list.
4. `place` is the interesting one. Sketch:
 - Guard the placed status

- Check that lines isn't empty (else `CannotPlaceEmptyOrder`)
 - Compute the total via `result.try` (else propagate the underlying error)
 - Construct the placed order plus its `OrderPlaced` event
5. Don't share event-construction code across operations. Each event has a different shape, so build each one inline at the emission site; it's a handful of lines per operation.
 6. The kata 4 helpers carry over untouched: `no_modify_placed`, `non_empty_sku`, `positive_qty`, and `currency_matches` keep their signatures and bodies. They short-circuit before you build anything.

Walk-through

`new` is total: a tuple of the new draft order and an event list with one `OrderCreated`. `add_line` builds its event in the `Ok` branch only, since the `use <-` guards short-circuit on failure and a failure isn't a fact.

`place` is the chapter's showpiece (the code block in "result.try chaining" above): a zero-arg `use <-` for the status guard, a `case` for the empty-lines guard, and a `use t <- result.try` for the fallible total. Three failure layers stack independently; the placed-order record and its event come together only after every layer passes.

`total` is `pub` because callers want to display a running total before placing, and the scenario tests assert on expected totals. Exposing it doesn't break the aggregate boundary, since it's a pure read.

Scenario testing: commands as data

Once the aggregate has multiple operations that chain together, per-operation tests start looking the same:

build empty order → add line → add line → place → assert.

That's a lot of ceremony per assertion. The cleaner pattern lifts the inputs to data, runs them through a tiny engine, and lets the tests read as declarative scenarios.

The full file lives at `test/order_scenarios_test.gleam`; the core is about fifteen lines.

```

pub type OrderCommand {
  AddLine(sku: String, quantity: Int, unit_price: money.Money)
  Place
}

pub fn run(
  initial: order.Order,
  cmds: List(OrderCommand),
) -> Result(#(order.Order, List(order.OrderEvent)), order.OrderError) {
  list.try_fold(cmds, #(initial, []), apply_one)
}

fn apply_one(state, cmd) {
  let #(o, events_so_far) = state
  let step = case cmd {
    AddLine(sku, q, price) -> order.add_line(o, sku, q, price)
    Place -> order.place(o)
  }
  use #(o2, new_events) <- result.try(step)
  Ok(#(o2, list.append(events_so_far, new_events)))
}

```

`list.try_fold` walks the command list, threading (`order`, `events`) through each step. The first failure short-circuits the fold and bubbles out as `Error(...)`, while successful runs accumulate events in order.

Tests then read like specifications.

```

pub fn two_lines_then_place_emits_correct_events_test() {
  let oid = test_order_id()
  let cmds = [
    AddLine("WIDGET", 2, usd(50)),
    AddLine("GADGET", 1, usd(100)),
    Place,
  ]
  let assert Ok(#(_, events)) = run(empty_draft_order(), cmds)
  assert events == [
    order.LineAdded(oid, "WIDGET", 2, usd(50)),
    order.LineAdded(oid, "GADGET", 1, usd(100)),
    order.OrderPlaced(oid, usd(200)),
  ]
}

```

```

    ]
  }

  pub fn add_line_after_place_fails_test() {
    let cmds = [
      AddLine("WIDGET", 1, usd(50)),
      Place,
      AddLine("GADGET", 1, usd(50)), // fails because order is placed
    ]
    assert run(empty_draft_order(), cmds)
      == Error(order.CannotModifyPlacedOrder)
  }

```

Each scenario is one `let cmds = [...]` followed by one assertion, where the command sequence carries the meaning and the assertion locks it in. Fifty of these read in two minutes and you know exactly what the aggregate does.

The pattern scales because the set of failure scenarios is finite and easy to enumerate; once they all pass, you trust the aggregate's behavior rather than just its per-operation correctness.

Takeaway

The aggregate-as-pure-function trajectory reaches its natural endpoint:

$$\text{aggregate} \times \text{command} \rightarrow \text{Result}(\#(\text{new_aggregate}, \text{events}), \text{error})$$

That signature is the central abstraction every CQRS and event-sourcing framework reaches for, written here without a framework in about a hundred lines of Gleam, with the type system enforcing every invariant.

A richer system wouldn't return events to the caller. It would publish to a bus and let other bounded contexts subscribe, so `Order` stays unaware that `Shipping` exists yet shipping still happens because shipping listens for `OrderPlaced`. That decoupling earns events their place as a first-class concept.

Bonus: commands as a first-class concept

`OrderCommand` lives in test code rather than in the domain. That was deliberate; this kata is about events. In a fully event-sourced system, commands graduate to the domain:

```
// commands flow in from the boundary (HTTP, message queue, etc.)
// events flow out to the bus
pub fn handle(
    state: Order,
    cmd: OrderCommand,
) -> Result(#[Order, List(OrderEvent)], OrderError) {
    case cmd {
        AddLine(...) -> add_line(state, ...)
        Place -> place(state)
    }
}
```

That function is the aggregate's API; everything else is detail. CQRS frameworks like Axon, EventFlow, and akka-persistence-typed sit on this signature.

The kata doesn't need that abstraction yet, but the scenario engine you just wrote is the test-time twin of the production command handler, the same pattern wearing a different label.

What's next

Kata 6: **Repositories**. The aggregate has been a pure function so far; the next chapter introduces persistence. A use case will load an `Order` from storage, call `place`, save the result, and publish the events, and the domain code never learns what storage is.

That's where the layering from `00_introduction.md` starts earning its keep.

07. Kata 6: Repositories

Concept

The aggregate has stayed pure. State and a command go in, a new state and events come out. That work is worth nothing once the process restarts.

A repository turns an aggregate type into something you can load and save. The domain depends on the interface and the implementation lives in the infrastructure layer, so `order.gleam` stays ignorant of storage.

Use cases then test without infrastructure. A test places an order through the repo, loads it back, and checks the events. Storage stays pluggable on the other side, so you swap an in-memory dict today for Postgres next week, and the use-case code on top doesn't notice.

A repository belongs to an aggregate, not an entity. You save an `Order` and an `OrderLine` rides along inside it. The aggregate is the consistency unit, and the repo's contract is "give me back a valid one or fail."

What you load is what you trust

The repository's other job is to feed loaded data back through the smart constructors. A database row is raw fields, and turning those into a typed `Order` goes through `email.new`, `customer.new_id`, and friends, the same validation that keeps HTTP input honest. A corrupt row becomes `Error(CorruptRow(...))` rather than a thrown exception.

The Kata 1 ceremony compounds here. Every reconstruction trusts the type because every construction went through the smart constructor. No "but I got it from the database, so I trust it" backdoor exists.

The use case is the orchestrator

A use case is a function that does one thing the application supports, like "place an order." The shape is:

07. Kata 6: Repositories

load via repo → call domain → save via repo → return events

Every step has a typed error. The use case wraps repo errors and domain errors in a `PlaceOrderError` so callers know which layer broke, which is where the chaining patterns from earlier katas come together: `result.try` from Kata 3 and the wrap-the-cause idea from Kata 5.

The use case is the application boundary that HTTP, CLI, and message handlers translate into and out of. The domain sits beneath it. A test of a use case says “given these inputs, this is what happens” without spinning up any infrastructure.

New Gleam fundamentals

Records of functions

A Gleam record whose fields happen to be functions is still an ordinary record, and that record serves as an interface:

```
pub type OrderRepo {
  OrderRepo(
    find: fn(OrderId) -> Result(Order, RepoError),
    save: fn(Order) -> Result(Nil, RepoError),
  )
}
```

A `Customer` and an `OrderRepo` are the same kind of value. The fields differ and callers don't care:

```
let order = repo.find(id)
```

A type with function fields is the interface, without an `IFooRepository<Order>` or a DI container in sight.

Why we need OTP

Gleam has no globals or mutable fields. State that needs to survive across function calls has to live somewhere, and that somewhere is a process.

A process is a lightweight Erlang thread with a mailbox. You send messages and it handles them one at a time. The state is a value the process threads through its message loop, and outside the process you see replies rather than the state itself.

For a repository, the process holds a `Dict(OrderId, Order)` and the messages ask “find this id” or “save this order.”

OTP, just enough

The vocabulary stays compact.

- `Dict(k, v)` from `gleam/dict`, an immutable hash map. `dict.new()`, `dict.get(d, k) -> Result(v, Nil)`, `dict.insert(d, k, v) -> Dict(k, v)`.
- `Subject(msg)`, a typed mailbox, the address of a process. Hold a `Subject(Find(...) | Save(...))` to send messages to a particular actor.
- `actor.new(initial_state) |> actor.on_message(handler) |> actor.start` builds and starts the actor. Returns `Result(Started(Subject(msg)), StartError)`. The `Started.data` field is the subject outside callers use.
- The handler signature is `fn(state, msg) -> actor.Next(state, msg)`. Return `actor.continue(new_state)` to loop, or `actor.stop()` to terminate.
- `process.call(subject, timeout, builder)` does synchronous request/reply. The builder has type `fn(reply_subject) -> msg`. You embed a reply subject in the outgoing message, the actor sends its response there, and `process.call` blocks until that response arrives or the timeout expires.

A counter actor builds the same shape:

```
type Msg {
  Get(reply: Subject(Int))
  Set(value: Int, reply: Subject(Nil))
}

fn handle(state: Int, msg: Msg) -> actor.Next(Int, Msg) {
  case msg {
    Get(reply) -> {
      process.send(reply, state)
      actor.continue(state)
    }
    Set(value, reply) -> {
      process.send(reply, Nil)
      actor.continue(value)
    }
  }
}
```

Swap `Dict(OrderId, Order)` for `Int` and swap `Find/Save` for `Get/Set`, and you have a repository.

Wrapping the actor in the interface

The actor's `Subject(Msg)` is an implementation detail. The use case sees an `OrderRepo`. Closures bridge the two:

```
pub fn in_memory() -> Result(OrderRepo, actor.StartError) {
  use started <- result.try(
    actor.new(dict.new())
    |> actor.on_message(handle_msg)
    |> actor.start,
  )
  let pid = started.data
  Ok(OrderRepo(
    find: fn(id) { process.call(pid, 100, fn(reply) { Find(id, reply) }) },
    save: fn(o) { process.call(pid, 100, fn(reply) { Save(o, reply) }) },
  ))
}
```

Each function field closes over the actor's subject. Callers never know the actor exists.

Error wrapping across layers

When two layers can fail, the error type names the layer:

```
pub type PlaceOrderError {
  RepoFailed(RepoError)
  DomainFailed(OrderError)
}
```

The conversion happens via `result.map_error`, which relies on a Gleam fact that's easy to miss: constructors are functions. When you write `pub type PlaceOrderError { RepoFailed(RepoError) | ... }`, `RepoFailed` does double duty. It works as a pattern in a `case` expression to destructure (`case e { RepoFailed(inner) -> ... }`), and it also works as a value of type `fn(RepoError) -> PlaceOrderError` that fits anywhere a function does.

`result.map_error` has signature `fn(Result(a, e1), fn(e1) -> e2) -> Result(a, e2)`. Given a `Result` and a function from the old error type to the new one, it applies that function on the `Error` branch and returns a `Result` whose error type is whatever the function returned.

Pass `RepoFailed` (a function `fn(RepoError) -> PlaceOrderError`) and the result's error type becomes `PlaceOrderError`:

```
let r1: Result(Order, RepoError)      = repo.find(id)
let r2: Result(Order, PlaceOrderError) = r1 |> result.map_error(RepoFailed)
//                                     RepoFailed is the function ^
```

The original `RepoError` survives inside the `RepoFailed` wrapper, so the cause is intact, and callers pattern-match on `RepoFailed(_)` versus `DomainFailed(_)` to see which side broke.

`result.try` only chains when the inner `Result` and the outer function's return type share an error type. Drop the `map_error` and the repo gives you `Result(_, RepoError)` while the outer function returns `Result(_, PlaceOrderError)`, so the types refuse to line up. `map_error(RepoFailed)` makes the chain type-check.

Task

Add an accessor to `src/order.gleam` so the repo can extract the ID from an order to use as a dict key:

```
pub fn id(order: Order) -> OrderId {
  order.id
}
```

Create `src/order_repo.gleam` exposing:

```
pub type RepoError {
  NotFound
  // add more if you discover them
}

pub type OrderRepo {
  OrderRepo(
    find: fn(OrderId) -> Result(Order, RepoError),
    save: fn(Order) -> Result(Nil, RepoError),
  )
}
```

```
pub fn in_memory() -> Result(OrderRepo, actor.StartError) {
  todo
}
```

Create `src/place_order.gleam` exposing:

```
pub type PlaceOrderError {
  RepoFailed(RepoError)
  DomainFailed(OrderError)
}

pub fn run(
  repo: OrderRepo,
  id: OrderId,
) -> Result(#(Order, List(OrderEvent)), PlaceOrderError) {
  todo
}
```

Wire `in_memory` to an actor that holds a `Dict(OrderId, Order)`. The actor handles `Find` and `Save` messages, and the use case loads, places, and saves.

Tests go in `test/order_repo_test.gleam` (round-trip the repo) and `test/place_order_test.gleam` (run the use case end-to-end against an in-memory repo).

Hints: what to do

1. Design the `OrderRepo` type before you write any code. `find` and `save` are the minimum; add `delete` or `list_for_customer` when a test asks for them and skip them until then.
2. `RepoError` is the next type to pin down. `NotFound` is obvious. Beyond that, `StorageError(reason)` covers network or disk failures and `CorruptRow(reason)` covers “I have a row but cannot reconstruct an `Order`.” Keep the type compact and grow it as you go.
3. `handle_msg` is the only real implementation work in `order_repo`. Match on `Find(id, reply)` or `Save(o, reply)`, run the matching dict op, send the answer with `process.send(reply, ...)`, then return `actor.continue(new_state)`. Each branch runs about five lines.
4. Put the actor’s state type in the handler signature: `fn handle_msg(state: Dict(OrderId, Order), msg: Msg) -> actor.Next(Dict(OrderId, Order), Msg)`. The signature is verbose, and the compiler keeps you honest.

5. `place_order.run` chains these steps:

- `use order <- result.try(repo.find(id) |> result.map_error(RepoFailed))`
- `use #(placed, events) <- result.try(order.place(order) |> result.map_error(DomainFailed))`
- `use _ <- result.try(repo.save(placed) |> result.map_error(RepoFailed))`
- `Ok(#(placed, events))`

6. For tests, write a helper that creates a repo and seeds it with an order. Most tests then set up state, call the use case, and assert against the events.

7. `process.call` blocks the calling process until the actor replies or times out. 100ms is plenty for tests.

8. `result.replace_error(NotFound)` is the cleanest way to turn the `Result(Order, Nil)` from `dict.get` into a `Result(Order, RepoError)`.

Walk-through

The actor's message type does the design work. Each request becomes a variant, and each variant carries a typed reply `Subject` so the actor knows where to send the answer:

```
pub type Msg {
  Find(id: OrderId, reply_to: Subject(Result(Order, RepoError)))
  Save(order: Order, reply_to: Subject(Result(Nil, RepoError)))
}
```

The reply types differ by message. The compiler refuses to let `Find` ever reply with `Nil` or `Save` with an `Order`, so the wiring stays honest without runtime checks.

`handle_msg` is then small:

```
fn handle_msg(store, msg) {
  case msg {
    Find(id:, reply_to:) -> {
      let r = store |> dict.get(id) |> result.replace_error(NotFound)
      process.send(reply_to, r)
      actor.continue(store)
    }
    Save(order:, reply_to:) -> {
      let new_state = dict.insert(store, order.id(order), order)
      process.send(reply_to, Ok(Nil))
      actor.continue(new_state)
    }
  }
}
```

```

    }
  }
}

```

`result.replace_error(NotFound)` swaps the `Nil` from `dict.get` for the repo's vocabulary in one call, since the missing-key case carries no detail worth preserving.

`in_memory` starts the actor and bridges its subject into the `OrderRepo` record, so callers see two function fields and never learn an actor exists:

```

pub fn in_memory() -> Result(OrderRepo, actor.StartError) {
  use started <- result.try(
    actor.new(dict.new())
    |> actor.on_message(handle_msg)
    |> actor.start,
  )
  let pid = started.data
  Ok(OrderRepo(
    find: fn(id) { process.call(pid, 100, fn(reply) { Find(id, reply) }) },
    save: fn(o)  { process.call(pid, 100, fn(reply) { Save(o, reply) }) },
  ))
}

```

The `fn(reply) { Find(id, reply) }` shape is `process.call`'s contract: it hands you a fresh reply subject, you embed it in the outgoing message, and `process.call` blocks until the actor sends a response there or 100ms passes.

`place_order.run` is the payoff:

```

pub fn run(repo, id) {
  use order <- result.try(repo.find(id) |> result.map_error(RepoFailed))
  use #(placed, events) <- result.try(
    order.place(order) |> result.map_error(DomainFailed),
  )
  use _ <- result.try(repo.save(placed) |> result.map_error(RepoFailed))
  Ok(#(placed, events))
}

```

Three `result.try` lines, each wrapping the layer-specific error in a `PlaceOrderError` variant. The function reads top-to-bottom like a checklist: load, transition, save, return. Nothing here knows the repo is an actor or that the actor holds a `Dict`, so swapping in a SQLite repo (kata 9) doesn't move a line.

The `map_error(RepoFailed)` calls do the type-juggling the “constructors as functions” detour set up. Without them the inner `Result(_, RepoError)` wouldn’t fit a chain expecting `Result(_, PlaceOrderError)`.

Critique

The 100ms timeout fits an in-memory dict and nothing else. A real I/O repo would take the timeout as config, since “long enough for the slow case, short enough that hung backends don’t pile up callers” is a deployment decision rather than a code one.

The actor itself is a teaching scaffold. It serializes every request through one mailbox, which a `Dict` needs and a SQL connection pool already provides. A SQLite or Postgres repo drops the actor entirely; the in-memory dict has nowhere else to live, so it borrows one.

`StorageError(String)` is a placeholder. A real repo distinguishes “connection lost,” “constraint violated,” and “decode failed” because callers want to react differently. Grow the variant set as the production adapter forces the cases.

`list_all` is `pub` only because integration tests want to read everything back. Production repos almost never expose it directly, since returning the whole table is an outage waiting to happen. Filtered or paginated queries earn the field; an unbounded list doesn’t.

`order.id(order)` is the one accessor that crosses the aggregate’s opaque-type wall for the repo’s benefit, and it’s the right size: the repo needs a key to store under, and an explicit accessor is cheaper than exposing the record. Kata 9 extends the same idea (`snapshot/restore`) when the repo needs the full state.

Takeaway

The aggregate stayed pure. `order.place` still takes an `Order` and returns `Result( #(Order, List(OrderEvent)), OrderError)`, the same signature it had after kata 5. Nothing in `order.gleam` learned that storage exists.

What the repository bought, in one chapter: a use case that takes an interface and swaps adapters in kata 9 without moving, tests that build a fresh in-memory backend per case without mocks, and layer errors that stay distinct all the way to the HTTP boundary where `RepoFailed(NotFound)` becomes a 404 and `DomainFailed(CannotPlaceEmptyOrder)` becomes a 422.

07. *Kata 6: Repositories*

The use case is the application layer the rest of the book builds on. HTTP handlers (kata 8) translate requests into use-case calls; bounded contexts (kata 7) react to the events use cases return. Its signature reads like a sentence about what the application does.

What's next

Kata 7 introduces a second bounded context, Shipping, that reacts to Ordering's events without either side importing the other's internals. The repository pattern from this chapter carries straight through: `ShipmentRepo` is the same record-of-functions on the same actor scaffold, with the message type adapted to shipping's queries.

08. Kata 7: Bounded Contexts

Concept

So far you've worked inside one bounded context, **Ordering**, with `Order` as the aggregate, `place_order` as the use case, and `OrderRepo` as the repo. Everything in `src/` belongs to that context.

Real systems grow more. Warehouse ships the placed orders while Marketing segments customers by lifetime value and Finance reconciles payments. Each team owns its own data and vocabulary, and moves at its own pace.

Each becomes its own bounded context with its own model. They share concepts (a customer in Ordering is the same human as a customer in Marketing), but the *data* each context holds about that customer diverges. Force them into one type and the model dies fast:

- A `Customer` type with `name`, `email`, `segment`, `ltv`, `last_campaign`, `default_address`, `loyalty_tier`, `risk_flags`...
- Changes ripple through every team, reads pull fields nobody on this team cares about, and migrations turn into multi-team negotiations.
- The privacy story collapses the moment Marketing's PII fields end up in Ordering's fixtures.

"Customer in Ordering" and "Customer in Marketing" are different concepts that happen to share an ID. Each gets its own type and its own storage, and they talk through events.

The thesis kata 7 demonstrates

Ordering doesn't know Shipping exists.

Shipping reacts to Ordering's events. Ordering just emits them.

Events do the integration work, and neither context shares a type with the other.

In code, `src/shipping/*` imports `order.OrderEvent` and `order.OrderId`, while `src/order.gleam` imports nothing from `shipping/`. Grep proves the asymmetry.

New Gleam fundamentals

Kata 7 is about *organization*, so the new machinery is minimal.

Folders as namespace boundaries

Gleam's module system is filesystem-driven. `src/shipping/shipment.gleam` exports a module called `shipping/shipment`. Other code imports it as:

```
import shipping/shipment.{type Shipment}
```

The folder *is* the namespace. Living inside `shipping/` is what makes a file part of the Shipping bounded context, and no other ceremony applies.

Type aliases for function types

To name a function type, say the event handler signature, Gleam lets you write:

```
pub type OrderEventHandler =  
  fn(order.OrderEvent) -> Result(Nil, ShipmentError)
```

Now `handler: OrderEventHandler` stands in for the long signature. The alias is optional and worth it only when it clarifies the calling code.

That's the new mechanics. The rest is patterns you already use: opaque types, smart constructors, records of functions, modules.

Task: one file per step

Create five new files. Each does one job, and reading the kata top-down matches reading the files in order.

1. src/shipping/shipment.gleam: the Shipping aggregate

```

pub opaque type ShipmentId {
  ShipmentId(value: String)
}

pub type ShipmentStatus {
  Pending
  Shipped
  Delivered
}

pub opaque type Shipment {
  Shipment(
    id: ShipmentId,
    order_id: order.OrderId,
    status: ShipmentStatus,
  )
}

pub type ShipmentError {
  EmptyShipmentId
  CannotShipNonPending
  CannotDeliverNonShipped
}

pub fn new_id(raw: String) -> Result(ShipmentId, ShipmentError)
pub fn new(id: ShipmentId, order_id: order.OrderId) -> Shipment
pub fn id(s: Shipment) -> ShipmentId
pub fn order_id(s: Shipment) -> order.OrderId
pub fn mark_shipped(s: Shipment) -> Result(Shipment, ShipmentError)
pub fn mark_delivered(s: Shipment) -> Result(Shipment, ShipmentError)

```

The shape mirrors `Order`. This kata skips events because kata 7 focuses on the cross-context flow, and kata 5 already covered how aggregates emit events.

2. src/shipping/shipment_repo.gleam: the Shipping repository

```

pub type RepoError {
  NotFound
}

pub type ShipmentRepo {
  ShipmentRepo(
    find: fn(ShipmentId) -> Result(Shipment, RepoError),
    save: fn(Shipment) -> Result(Nil, RepoError),
    find_by_order: fn(order.OrderId) -> Result(Shipment, RepoError),
  )
}

pub fn in_memory() -> Result(ShipmentRepo, actor.StartError)

```

The shape mirrors `OrderRepo` and adds one new `find_by_order` query, because the use case asks “did this order already produce a shipment?”, which is a lookup by `order_id` rather than by `shipment_id`. Repos expose what their callers need.

3. src/shipping/handle_order_placed.gleam: the cross-context handler

```

pub type HandleError {
  RepoFailed(shipment_repo.RepoError)
  ShipmentFailed(shipment.ShipmentError)
  AlreadyShipped
}

pub fn run(
  repo: ShipmentRepo,
  fresh_id: ShipmentId,
  event: order.OrderEvent,
) -> Result(Nil, HandleError)

```

The handler inspects the event and acts only on `OrderPlaced`. It asks the repo whether a shipment already exists for this order, since the bus can replay events and the handler has to stay idempotent. If none exists, it builds a fresh `Shipment` via `shipment.new` and saves it through `repo.save`.

This module imports `order` types (`OrderEvent`, `OrderId`) along with `shipping/shipment` and `shipping/shipment_repo`. It’s the one place where Ordering and Shipping meet.

4. test/shipping/shipment_test.gleam: Shipping aggregate tests

Standard aggregate tests cover construction, the happy-path transitions, and the error branches when the status is wrong.

5. test/shipping/handle_order_placed_test.gleam: the integration spec

One test proves the cross-context flow works:

```
pub fn order_placed_creates_shipment_test() {
  let assert Ok(order_repo) = order_repo.in_memory()
  let assert Ok(ship_repo) = shipment_repo.in_memory()

  // Place an order via Ordering
  let oid = test_order_id()
  let #(o, _) = order.new(oid, test_customer_id())
  let assert Ok(#(o2, _)) = order.add_line(o, "WIDGET", 1, usd(100))
  let assert Ok(_) = order_repo.save(o2)
  let assert Ok(#(_, events)) = place_order.run(order_repo, oid)

  // Dispatch the events to Shipping
  let assert [order_placed] = events
  let assert Ok(sid) = shipment.new_id("SHIP-001")
  let assert Ok(Nil) = handle_order_placed.run(ship_repo, sid, order_placed)

  // Shipment exists
  let assert Ok(s) = ship_repo.find_by_order(oid)
  assert shipment.order_id(s) == oid
}
```

The test runs a full vertical slice of two contexts collaborating, and no module here imports anything *from* shipping/ into order or the reverse.

Hints: what to do

1. Build in the order listed: aggregate first (no dependencies), repo next (depends on the aggregate and the actor scaffold), handler last (pulls in the aggregate, the repo, and Ordering's OrderEvent).

2. `Shipment` looks like `Order` with the hard parts removed. There are no lines or totals, just an ID, an `OrderId` acting as a foreign key, and a status enum. State transitions check the current status.
 3. `shipment_repo.in_memory()` mirrors `order_repo.in_memory()` on the same actor scaffold; the message type grows one variant to support `FindByOrder`.
 4. Make the handler idempotent. Pattern-match the event. Anything that isn't `OrderPlaced` returns `Ok(Nil)` and bails. For `OrderPlaced`, call `find_by_order` first, because if a shipment already exists you return `Ok(Nil)` (or `Error(AlreadyShipped)`, your call). Only `save` when nothing's there.
 5. Pass the `ShipmentId` in from outside. Generating IDs inside the handler makes it nondeterministic and harder to test, so let the composition root or the test supply a fresh one.
 6. Imports tell the story. When you finish, check:
 - `src/shipping/handle_order_placed.gleam` should import `order.{type OrderEvent, OrderPlaced, type OrderId}`
 - `src/order.gleam` should have **zero** imports starting with `shipping/`
 - If you have to import the wrong direction, the design is upside down.
 7. For `mark_shipped / mark_delivered`: `case s.status { Pending -> Ok(Shipment(..s, status: Shipped)) ; _ -> Error(CannotShipNonPending) }`, which is the same pattern you used in `Order.place`.
-

Walk-through

The directory structure *is* the bounded context. There's no `Context` type and no `BoundedContext` annotation declaring boundaries in a config file. You have `src/shipping/`, and the folder is the boundary. Grep and dependency-graph tools see the boundary without further input.

`handle_order_placed` is the only file with both worlds in scope. Its imports tell the integration story:

```
import order.{type OrderEvent, OrderPlaced}    // ← Ordering's published events
import shipping/shipment.{type ShipmentId}      // ← Shipping's own types
import shipping/shipment_repo.{type ShipmentRepo, NotFound}
```

The module sits *inside* the Shipping context (path: `shipping/`) but *translates* events from Ordering. This file names the cross-context relationship in code, in one place. Move it and the relationship moves with it.

Nothing touches Ordering. `src/order.gleam` gains no new imports and no new methods or callbacks. Ordering keeps emitting `OrderPlaced` events; whether anyone's listening is none of its business. Delete the `shipping/` directory and `src/order.gleam` still compiles and works.

The test asserts the integration without testing the bus. No event bus type appears. The test calls `place_order.run`, takes the events back, and hands them to the Shipping handler. Those two lines *are* the bus. A fancier one with async delivery, multiple subscribers, or retry on failure is a separate project, and the kata works fine without it.

The `find_by_order` check before saving makes the handler safe to replay. Real event delivery is at-least-once, and handlers that aren't idempotent end up creating duplicates, so putting the check in the handler buys idempotency at the application layer for free.

Critique

Where do `money` and `email` live? Both contexts use them, so there are a couple of honest answers:

- **Shared kernel** (current setup): they sit at the top level (`src/email.gleam`, `src/money.gleam`) and both contexts import them. Good call when the value objects are small, stable, and behave the same wherever they go.
- **Duplicate**: each context defines its own. Good call when contexts evolve at different rates or need different behavior on the same concept (Marketing might tack `tax_jurisdiction` onto its money type for compliance).

A kata sticks with the shared kernel. A real system duplicates as soon as you feel the pull.

The kata ships without an anti-corruption layer. The handler imports `order.OrderEvent` directly, so if Ordering's event vocabulary changes the handler breaks at compile time. That's a feature when both contexts share a team who can fix both sides in one PR, and a bug when the teams are independent. In the independent case, add a tiny adapter:

```
// src/shipping/order_event_adapter.gleam
pub type ShippingTrigger {
  TriggerForOrder(OrderId) // shipping's own vocabulary
}

pub fn from_order_event(e: order.OrderEvent) -> Option(ShippingTrigger) {
  case e {
    OrderPlaced(id, _) -> Some(TriggerForOrder(id))
    _ -> None
  }
}
```

```
}  
}
```

Now `handle_order_placed` deals only in `ShippingTrigger`. Ordering can rename, add, or remove event variants and only `order_event_adapter.gleam` updates. That’s an Anti-Corruption Layer in eight lines, and you should add it once you need it, not before.

There’s no real event bus. The “bus” is the composition root iterating events and calling handlers, which suits one process with synchronous handlers and runs out of room when async delivery, persistent retry, or dead-letter queues come up. The fix is a real broker (RabbitMQ, NATS, Kafka, BEAM distribution); application code doesn’t move, only the wiring.

`Shipment` doesn’t emit events of its own. Adding `ShipmentShipped` would feed tracking and Finance reconciliation, but the kata skips it to keep the focus on cross-context flow instead of redoing kata 5.

Process managers and sagas are also out of scope. Coordinating `OrderPlaced` → `CreateShipment` → `ChargeCard` → `ConfirmShipment` across contexts is a saga’s job; the pattern here is one idempotent handler per event, with no orchestration on top.

Takeaway

Two bounded contexts now live in one process, communicating via events along an asymmetric dependency arrow the file tree makes visible. One file (the handler) names both contexts and is idempotent; a test exercises the cross-context flow with no mocks. Adding Marketing means `src/marketing/`; adding Finance means `src/finance/`. The shape repeats.

The lessons compound. The boundary lives in the dependency graph rather than in documentation, since comments and team agreements drift while imports don’t. Events are the integration contract, named and versioned, and changing one is a public API change while changing a domain type is a private refactor. “Same word, different model” is the right answer: contexts share IDs, not data. A Shipping service in another datacenter is the same code with a different transport, which is why monoliths and microservices with bounded contexts swap more easily than the marketing language suggests.

09. Kata 8: Composition Root + HTTP Boundary

Concept

The functional core is done: pure aggregates, smart constructors, events as data. The application layer sits on top: repositories and use cases that orchestrate them. What's left is the shell, which takes input from outside (HTTP, say) and turns the result back into a response.

This kata wires up:

1. A composition root (`src/main.gleam`), the one file that knows how to construct concrete adapters and assemble them into a running app.
2. An HTTP boundary, a thin handler that turns requests into use-case calls, then turns the typed result into a status code and JSON body.
3. The DI pattern in working code, a `Deps` record built once at startup and handed to the request handler through closure capture.

The shell translates; the core decides. Decisions live in pure functions you've already written (`order.place`, `place_order.run`), and the shell parses strings into typed values (`path` → `OrderId`), routes requests to use-case calls, and translates typed results into status codes and bodies.

A business rule in the handler means you've drifted out of the shell and back into the core, and the rule belongs further down.

Why this is the FCIS payoff chapter

Functional core / imperative shell is a slogan until you see the shell in code. Your `place_order.run` signature looks like this:

```
pub fn run(repo, id) -> Result(#[Order, List(OrderEvent)], PlaceOrderError)
```

That signature is complete. The handler doesn't need to know anything beyond "call this, get one of these shapes back, translate":

09. Kata 8: Composition Root + HTTP Boundary

```
case place_order.run(repo, id) {
  Ok(_)                                -> 200 OK
  Error(RepoFailed(NotFound))          -> 404 Not Found
  Error(DomainFailed(CannotPlaceEmptyOrder)) -> 422 Unprocessable
  Error(DomainFailed(CannotModifyPlacedOrder)) -> 409 Conflict
  Error(_)                             -> 500 Internal Server Error
}
```

Every variant the use case can return becomes one HTTP response, and the handler does nothing else. The compiler enforces exhaustiveness: add a new variant to `PlaceOrderError`, forget the HTTP mapping, and the build breaks at the `case` instead of silently returning 500 in production.

Types-as-proof at the boundary stops the shell from quietly ignoring a failure mode the core cares about.

New Gleam fundamentals

Wisp: the HTTP framework

Wisp is the standard Gleam web framework. The shape you'll lean on:

```
import wisp.{type Request, type Response}

// A handler is just a function:
fn handle(req: Request) -> Response { ... }

// Routing: pattern-match on path segments and method
case wisp.path_segments(req), req.method {
  ["orders", id, "place"], http.Post -> ...
  _, _ -> wisp.not_found()
}
```

Response constructors come in two flavours: named helpers for the common codes (`wisp.ok()`, `wisp.not_found()`, `wisp.bad_request(...)`, `wisp.unprocessable_content()`, `wisp.internal_server_error()` and `wisp.response(status)` for any code that doesn't have one (`wisp.response(409)` for Conflict, for example). JSON bodies go through `wisp.json_response(json, status)`.

The exact set of named helpers drifts between Wisp versions, so the [Wisp docs](#) are the source of truth; this chapter pins down only the ones the kata uses.

Mist: the underlying HTTP server

Wisp doesn't speak HTTP itself; it delegates to Mist. The wiring at the top of `main`:

```
import mist
import wisp/wisp_mist

pub fn main() {
  let secret = wisp.random_string(64) // session signing key
  let assert Ok(_) =
    wisp_mist.handler(handle_request, secret)
    |> mist.new
    |> mist.port(8080)
    |> mist.start
  process.sleep_forever()
}
```

Exact function names drift between Mist versions, so the docs are the source of truth.

Wisp/simulate: testing in-process

You don't need a running server to test handlers. `wisp/simulate` builds request values and feeds them through your handler function:

```
import wisp/simulate

let req = simulate.request(http.Post, "/orders/ORDER-001/place")
let response = handle_request(req)
assert response.status == 200
```

The shell is testable because handlers are functions from request to response, with impurity tucked inside the `Deps` closure and the in-memory repo giving you deterministic state.

The `Deps` closure pattern (DI in chill mode)

The handler needs the repository, and the repository can't be a global because each test wants its own clean copy. So composition time assembles the handler from the `deps`:

```
pub fn router(deps: Deps) -> fn(Request) -> Response {
  fn(req) {
```

09. Kata 8: Composition Root + HTTP Boundary

```
case wisp.path_segments(req), req.method {
  ["orders", id, "place"], http.Post ->
    place_order_handler.run(deps, id)
  _, _ -> wisp.not_found()
}
```

`router` returns a function that closes over `deps`, and `Mist` serves that function. Each test builds different `deps` and gets a different router, with the closure acting as the container.

Task

Add `wisp` and `mist` as dependencies:

```
gleam add wisp mist
```

Create the source files and tests below.

1. `src/main.gleam`: the composition root

```
pub fn main() -> Nil
```

Responsibilities:

- Construct the in-memory `OrderRepo` (and anything else the app needs).
- Bundle into a `Deps` record.
- Build the router from `Deps`.
- Start `Mist` on a port.

Only this file knows the concrete adapters. Below it the use case takes an interface, never the concrete type.

2. src/web/router.gleam: request to response dispatcher

```
pub type Deps {
  Deps(order_repo: OrderRepo)
}

pub fn handle(deps: Deps, req: Request) -> Response
```

Pattern-match path segments and method; dispatch to handlers; return `wisp.not_found()` for unmatched routes.

3. src/web/place_order_handler.gleam: the one HTTP handler

```
pub fn run(deps: Deps, raw_id: String) -> Response
```

Steps inside `run`:

1. Parse `raw_id` $\rightarrow$ `OrderId` via `order.new_id`. On failure, `wisp.bad_request`.
2. Call `place_order.run(deps.order_repo, order_id)`.
3. Pattern-match the result. Translate every variant to an HTTP response.

4. test/web/place_order_handler_test.gleam

Use `wisp/simulate` to drive the handler in-process. Test cases:

- 200 on a successful placement (seed a placeable order, hit the endpoint, assert status + body)
- 400 on a malformed `OrderId` (e.g. empty path segment if your routing allows it)
- 404 when the order doesn't exist (`RepoFailed(NotFound)`)
- 422 on `CannotPlaceEmptyOrder`
- 409 on `CannotModifyPlacedOrder`

Endpoint contract

POST `/orders/:id/place`

200 OK	{ "order_id": "...", "total": "...", "events": [...] } }
400 Bad Request	bad order id format
404 Not Found	order not in repo
409 Conflict	order already placed
422 Unprocessable	domain rule violation (empty order, etc.)

500 Server Error unexpected

Hints: what to do

1. Build the Deps record first. It's one field now (`order_repo`). Growing it later costs nothing for anything that already takes `Deps`, the DI-bag pattern from the chill-DI conversation.
 2. The handler is a translation layer. Doing logic in there beyond “call use case, map result to HTTP” means you should push that logic into the use case. The shell decides response shapes, never business outcomes.
 3. Exhaustive `case` saves you. Match on `Result(_, PlaceOrderError)` and the compiler refuses to let you forget a variant. A wildcard `_` belongs at the end for unmodeled storage failures (500), not as a substitute for handling a known error.
 4. Closure capture for DI. The router takes `Deps` and returns `fn(Request) -> Response`. Mist serves the returned function. Each test builds its own `Deps` and gets its own handler, so there's nothing to share or tear down.
 5. For the JSON body, use `gleam_json` (`gleam add gleam_json`). Build the body as `json.object([...])`, encode to a string, pass to `wisp.json_response(body, 200)`.
 6. Tests don't need Mist. `wisp/simulate.request(method, path)` builds a request value; pass it to your handler directly and assert on the response. The handler is a function; call it like one.
 7. `main.gleam` ends with `process.sleep_forever()`. Mist runs in a supervised process, so without something to keep `main` alive, the OS exits immediately.
-

Walk-through

The composition root:

```
let assert Ok(repo) = order_repo.in_memory()
let deps = Deps(order_repo: repo)
let handle = router.handle(deps, _) // closes over deps
```

The third line uses Gleam's `_` placeholder for the request. `router.handle` takes `(deps, req)`; Mist wants `fn(req)`. The `_` builds a closure that bakes in `deps` and leaves `req` open, the same capture trick you wrote in `order_repo` with `Save(order, _)`.

The handler’s case-on-result is the table from the kata description:

```
case place_order.run(deps.order_repo, oid) {
  Ok(#(_, events))           -> { /* 200 + JSON */ }
  Error(RepoFailed(NotFound)) -> wisp.not_found()
  Error(DomainFailed(CannotPlaceEmptyOrder)) -> wisp.unprocessable_content()
  Error(DomainFailed(CannotModifyPlacedOrder)) -> wisp.response(409)
  Error(DomainFailed(_))      -> wisp.unprocessable_content()
  Error(RepoFailed(_))        -> wisp.internal_server_error()
}
```

Each domain failure maps to the most informative HTTP code:

Domain error	HTTP	Why
CannotPlaceEmptyOrder	422	The request was syntactically OK, but the resource state forbids the action
CannotModifyPlacedOrder	409	The action conflicts with the resource’s current state
Other domain errors	422	Catch-all for “rules violated”
RepoFailed(NotFound)	404	No such resource
Other repo errors	500	Storage broke; client did nothing wrong

Every domain-knowable failure lives in the type system, so the boundary translates each one into something meaningful, and anything unknown falls through to 500. The boundary stays thick at the edges and thin in the middle, translating what it can and deciding nothing.

Kicking the tires with Hurl

`wisp/simulate` tests the handler in-process, which is fast and hermetic but doesn’t actually start the server. Once `gleam run` is listening on `:8080`, the cleanest way to exercise it from outside is [Hurl](#), a text-file HTTP runner that reads like raw requests with assertions stapled on.

Install it once:

09. Kata 8: Composition Root + HTTP Boundary

```
brew install hurl      # macOS
# or: cargo install hurl
```

The repo ships a small set of `.hurl` files under `dev/hurl/`, one per endpoint plus a `flow.hurl` that chains `create`, `add lines`, `place`, `read`, and `re-place` (asserting 409 on the second placement). Variables live in `dev/hurl/vars.env`.

Start the server in one terminal:

```
gleam run
```

In another, fire a single request with full output:

```
hurl --include --variables-file dev/hurl/vars.env dev/hurl/create.hurl
```

Or run the full scenario in `--test` mode (suppresses request/response bodies, prints pass/fail per file):

```
hurl --test \
  --variables-file dev/hurl/vars.env \
  --variable order_id=ORDER-$(date +%s) \
  dev/hurl/flow.hurl
```

A fresh `order_id` per run avoids the second invocation tripping on the placed-state guard from the previous run; restarting the in-memory server has the same effect.

`dev/hurl/errors.hurl` walks the 400 / 404 / 422 / 409 branches. When a use-case error variant moves or the handler's `case` arm drifts, one of those assertions starts failing before any reader runs into the bug.

Two further scripts in the same directory go beyond smoke testing:

- `workout.hurl` is a full state-machine traversal across four customers (happy path, currency mismatch, empty place, modify after place), with rich `jsonpath` assertions on order status, line count, and SKU membership.
- `pool-party.sh` fires `N` copies of `flow.hurl` in parallel via `xargs -P`, each with a unique `order_id`. Every flow runs six sequential requests; if the OTP actor ever interleaves them across order ids, the 409-on-re-place assertion catches it. 200 flows / 32 in flight clears in about a second on a laptop.

Critique

The POST puts all inputs in the URL path, so there's no JSON request body. Real APIs do take them, and a decoder per endpoint translates `Result(InputStruct, JsonError)` into either a 422 with details or a use-case call. The shape doesn't change; it's another translation layer in the shell. Wisp + `gleam_json` cookbook material covers it.

Auth, content negotiation, and observability are all absent. Wisp ships middleware for sessions and CSRF; OpenTelemetry packages exist for real instrumentation; an `Accept` header check dispatches to a different formatter. Each one stacks on top of the boundary without moving it, so the kata leaves them out.

`Deps` holds one field, which would normally be a smell. The bag stays because the moment a real app adds a `clock`, an `event_bus`, or a `customer_repo`, it absorbs them without rippling through every signature.

Persistence is in-memory, so restarts drop every order. Kata 9 swaps in SQLite behind the same `OrderRepo` interface; nothing above the adapter moves. That substitution is the payoff kata 6 promised, and the chapter on it is where the promise lands in working code.

Takeaway

The vertical slice runs end to end:

HTTP Request

```
src/main.gleam           ← composition root: builds deps, starts server

src/web/router.gleam     ← request → handler dispatch

src/web/place_order_handler.gleam ← parse, call use case, translate result

src/place_order.gleam    ← use case: load → place → save

src/order_repo.gleam     ← in-memory adapter (or Postgres in prod)

src/order.gleam          ← pure aggregate
```

09. Kata 8: Composition Root + HTTP Boundary

Each layer is independently testable and carries its own type vocabulary. The dependency arrows all point inward, and the composition root is the one file that constructs concrete adapters and hands them down. Hexagonal architecture in working code amounts to a handful of files and a record of values, in place of six packages and an annotation-driven container.

10. Kata 9: SQLite Repository

Concept

`OrderRepo` keeps `find` and `save` as fields on a record so that the implementation can change without touching the use case. The only implementation so far is an in-memory actor, so restart the process and every order vanishes.

Write `pub fn sqlite(connection) -> OrderRepo` against the same `OrderRepo` interface, backed by a SQLite database. No other file in the system changes. `place_order.run` keeps calling the same record, and so do the handler and the existing `in_memory`-based tests.

That swap, with no other code aware of it, is the repository pattern in concrete form. If you find yourself touching the use case to make this work, the design drifted somewhere upstream and that's where the fix belongs.

Why SQLite for a learning project

SQLite needs no daemon and no port, only a file path or `:memory:` for tests. You write `CREATE TABLE`, bind parameters, decode rows. The lessons transfer to Postgres with a different library (`pog` where you'd use `sqlight`) and the same testing model. Per-test isolation comes from calling `sqlight.open(":memory:")` in the test itself, which removes the fixture-and-rollback dance. File-based storage in production means backups are `cp`, which suffices for an app with one writer.

SQLite handles concurrent writers across processes badly. The kata sidesteps the issue; production cannot. One process with many readers and one writer is fine; once you need many writers across processes, reach for Postgres.

The interesting design problem

`Order` is opaque. `OrderLine` is private to `order.gleam`, so the type doesn't exist outside the module. When the SQLite adapter loads a row from the database, it can't construct an `Order` from raw fields. There is no public constructor that takes `(id, customer_id, lines, status)`. The adapter has options.

10. Kata 9: SQLite Repository

1. Replay the public API on load, calling `order.new(id, cid)`, then `order.add_line(...)` per line, then `order.place(...)` if the status was `Placed`. The public API emits events (`OrderCreated`, `LineAdded`, `OrderPlaced`), so loading from disk emits fake events as if the actions were happening fresh, which disqualifies the approach.
2. Punch a rehydrate back door in `order.gleam` that the persistence layer uses and application code doesn't. Add pub type `OrderSnapshot { ... }` with pub fn `snapshot(o) -> OrderSnapshot` and pub fn `restore(s) -> Order`. The snapshot exposes internal state in a controlled public shape without leaking the private types. The repo serializes the snapshot to JSON, stores it, decodes it back, and calls `restore`.

Option 2 is what every production system using opaque aggregates with state-stored persistence ends up doing. The public API enforces business invariants on transitions; the snapshot/restore pair handles persistence. That separation between transition rules and bytes on disk is the lesson of this kata, with SQL as the vehicle.

New Gleam fundamentals

sqlight: SQLite from Gleam

```
import sqlight

// Open
let assert Ok(conn) = sqlight.open(":memory:") // or "file:orders.db"

// DDL: no params, no result
let assert Ok(Nil) = sqlight.exec(
  "CREATE TABLE orders (id TEXT PRIMARY KEY, data TEXT NOT NULL)",
  on: conn,
)

// Parameterized query: INSERT/UPDATE/DELETE
let assert Ok(_) = sqlight.query(
  "INSERT INTO orders (id, data) VALUES (?, ?)",
  on: conn,
  with: [sqlight.text("ORDER-001"), sqlight.text(json_blob)],
  expecting: decode.success(Nil),
)
```



```
// SELECT with decoder
let assert Ok(rows) = sqlight.query(
  "SELECT data FROM orders WHERE id = ?",
  on: conn,
  with: [sqlight.text(id_string)],
  expecting: decode.field("data", decode.string),
)
```

`sqlight.text/int/float/bool/blob/null/nullable` build parameter values. `decode.string/int/field/...` from `gleam/dynamic/decode` builds the row decoder.

Errors come back as `sqlight.SqlightError(code, message, offset)`. Match on `code` for specific cases like `Constraint` or `Corrupt`.

gleam/json: encoding and decoding JSON

```
import gleam/json

// Encode
let body = json.object([
  #("id", json.string(id_str)),
  #("status", json.string("Placed")),
  #("lines", json.array(of: json.object, items: line_objects)),
])
let blob = json.to_string(body) // -> String

// Decode
import gleam/dynamic/decode

let order_decoder = {
  use id <- decode.field("id", decode.string)
  use status <- decode.field("status", decode.string)
  decode.success(#(id, status))
}
let assert Ok(parsed) = json.parse(blob, order_decoder)
```

The decoder type is `decode.Decoder(t)`, built by chaining `decode.field` calls inside a `use` block. The hints below walk through what an `Order`-shaped decoder looks like.

The LSP writes the boilerplate

The Gleam language server ships “Generate to-json function” and “Generate decoder” as code actions. With the cursor on a `pub type Foo { ... }` line, trigger code actions in your editor (. in VS Code, `<space>a` in Helix, `<leader>ca` in most nvim setups) and pick the generator. The LSP writes the encoder or decoder next to the type, populated from the fields it can see, and emits `todo` placeholders for nested types it hasn’t reached yet. Covering the full `Order` tree takes several invocations and a styling pass.

Visible code is the feature, not the cost

Other ecosystems hide encoders behind derive macros (`#[derive(Serialize)]`), annotations (`@JsonProperty`), or typeclass instances, all generated at compile time or via reflection. Gleam asks you to write the function (or have the LSP write it into your source). A few keystrokes more, paid back in: encoders you can read, grep, and step through; field renames that surface as compile errors instead of silent serialization drift; per-type encoding strategies without decorator soup; and one mechanism whether the bytes go to JSON, MessagePack, log lines, or fixtures.

The snapshot/restore back door pattern

Add these to `src/order.gleam`:

```
/// A serializable view of an Order's internal state. Public so the
/// persistence layer can read/write it; not for application code.
pub type OrderSnapshot {
  OrderSnapshot(
    id: OrderId,
    customer_id: CustomerId,
    lines: List(LineSnapshot),
    status: OrderStatus,
  )
}

pub type LineSnapshot {
  LineSnapshot(sku: String, quantity: Int, unit_price: Money)
}

pub fn snapshot(order: Order) -> OrderSnapshot {
  OrderSnapshot(
    id: order.id,
```

```

    customer_id: order.customer_id,
    lines: list.map(order.lines, fn(l) {
        LineSnapshot(sku: l.sku, quantity: l.quantity, unit_price: l.unit_price)
    }),
    status: order.status,
)
}

pub fn restore(snap: OrderSnapshot) -> Order {
    Order(
        id: snap.id,
        customer_id: snap.customer_id,
        lines: list.map(snap.lines, fn(l) {
            OrderLine(sku: l.sku, quantity: l.quantity, unit_price: l.unit_price)
        }),
        status: snap.status,
    )
}

```

`restore` is total. It assumes the snapshot is valid because the snapshot came from `snapshot`, which only produces valid snapshots from valid orders. The persistence layer makes sure the bytes round-trip. The domain doesn't re-validate.

The snapshot uses `LineSnapshot` instead of the private `OrderLine`. That keeps `OrderLine` private (no one outside `order.gleam` can construct one directly) while still giving the persistence layer a stable public shape to work with.

`Money`, `OrderStatus`, `OrderId`, and `CustomerId` are already public, and embed in the snapshot as-is.

Task

Add the dependency:

```
gleam add sqlight gleam_json
```

Then create one file and edit one:

1. Edit `src/order.gleam`: add the snapshot back door

Add `OrderSnapshot`, `LineSnapshot`, `pub fn snapshot`, and `pub fn restore` as described above. Around 30 new lines. The existing public API stays unchanged. The snapshot/restore pair is additive.

2. Create `src/order_repo_sqlite.gleam`: the SQLite adapter

```
pub fn sqlite(conn: Connection) -> Result(OrderRepo, RepoError)
```

Responsibilities:

1. Run schema setup (`CREATE TABLE IF NOT EXISTS orders (id TEXT PRIMARY KEY, data TEXT NOT NULL)`) on connection.
2. Build closures for `find` and `save`.
 - `find` queries by id. If no row, return `Error(NotFound)`. If a row, decode the `data` JSON into an `OrderSnapshot`, call `order.restore`, return `Ok(order)`.
 - `save` serializes via `order.snapshot` to JSON and runs `INSERT OR REPLACE INTO orders (id, data) VALUES (?, ?)`. On success it returns `Ok(Nil)`. Map sqlite errors to `RepoError`.
3. Return the `OrderRepo` record.

Around 60 to 80 lines, including the JSON encode/decode boilerplate.

3. Create `test/order_repo_sqlite_test.gleam`

Same shape as `order_repo_test.gleam` from kata 6. Open `:memory:` per test, build the repo, exercise the `find/save` round-trip, `NotFound`, and `overwrite`. The point is that the same assertions run against a SQLite-backed repo because the interface is the same.

Bonus: a `place_order` integration test using a SQLite repo. It's a copy of the existing integration test with the constructor swapped.

Hints

1. Build the schema before the queries. Write the `CREATE TABLE` and run it. Add a no-op `find` that always returns `NotFound` and a no-op `save` that always succeeds. The actor-shaped flow has to run end to end with the SQLite layer in place before query logic goes on top of a solid scaffold.
2. JSON encoding is verbose, and the verbosity is the point. `Money` becomes `{"amount": 150, "currency": "USD"}`; `Currency` and `OrderStatus` flatten to strings; `LineSnapshot` becomes an object. The full encoder runs roughly 30 lines of nested `json.object` calls.
3. Decoders mirror encoders. For each `json.object([("foo", ...)])` you wrote, there's a `decode.field("foo", ...)` you write. Build decoders for `Money`, `Currency`, `OrderStatus`, and `LineSnapshot`, then compose them into `OrderSnapshot`.
4. Use `INSERT OR REPLACE` for save. It handles insert-new and update-existing in one statement. The aggregate is the unit of save, so you replace the row.
5. Each `pub fn ..._test` opens its own `:memory:` connection. Sharing one connection means tests share state, and the suite goes flaky.
6. `sqlight.nullable(sqlight.text, optional_string)` is the binding helper for optional fields. Nothing in the current schema is null, but optional fields land in any real-world extension.
7. The hardest debugging path is silent JSON drift. If `snapshot |> encode |> string |> decode |> restore` doesn't round-trip exactly, load-time failures look mysterious. Write a unit test for the round-trip independent of SQLite: encode an order to JSON, decode it back, assert equality. It catches drift before SQL is involved.
8. `pub fn sqlite(conn)` takes an already-open connection rather than opening one itself. That keeps the repo testable (`:memory:` in tests, file connections in main) and lets the composition root own the connection lifecycle.

Walk-through

The schema fits in one table.

```
CREATE TABLE IF NOT EXISTS orders (
  id    TEXT PRIMARY KEY,
  data  TEXT NOT NULL -- JSON snapshot of OrderSnapshot
```

```
);
```

One row per order holds the serialized snapshot. Saving is `INSERT OR REPLACE`; finding is `SELECT data WHERE id = ?`. No joins or transactions yet.

`find` reads the row, decodes the JSON to an `OrderSnapshot`, and hands it to `order.restore`. `save` runs the inverse: `order.snapshot`, encode to JSON, `INSERT OR REPLACE`. The id comes from the `pub fn id` accessor added in kata 6.

If decode fails, the row is corrupt. Add `CorruptRow(reason: String)` to `RepoError` so callers see something other than a generic decode error.

The substitution at the composition root is one line:

```
// before
let assert Ok(repo) = order_repo.in_memory()

// after
let assert Ok(conn) = sqlight.open("file:orders.db")
let assert Ok(repo) = order_repo_sqlite.sqlite(conn)
```

`place_order.run`, the HTTP handler, and the use-case tests keep running unchanged. That swap is the repository pattern's whole point.

Critique

Schema migrations are absent. `CREATE TABLE IF NOT EXISTS` works for v1. The second version that adds a column needs a migration story: `ALTER TABLE`, a version table, and so on. Production apps reach for `dbmate` or `sqitch` for SQL files. In Gleam you can hand-roll a tracker that records applied migrations in a `_migrations` table. That work lives outside this kata.

Optimistic concurrency control is absent too. Two concurrent saves can race, and the second silently overwrites the first. For an actor-writes setup, which the in-memory actor model gives you by construction, that's fine. For multi-process writes, add a `version INT` column, check it on update, and fail with a `Conflict` variant when stale. Postgres and DynamoDB use the same pattern.

JSON storage is opaque to SQL. You can't write `SELECT * WHERE total > 100` because `total` lives inside the JSON blob. SQLite's `json_extract` lets you query into the JSON, but it's slow without expression indexes. If you need rich querying, denormalize, splitting into `orders` and `order_lines`

tables (the schema design covered in the chapter on **Shape** versus DB storage). The lesson on the rehydration back door doesn't change.

Streaming and batch operations stay out of scope. Each extension (`find_all`, `find_by_customer`, paginated reads) adds a field to `OrderRepo` and a closure in the constructor.

The connection is a shared resource. SQLite handles one writer. Multiple opens to the same file work, but write-serialization happens at the OS level. For production, wrap the `sqlight.Connection` in an actor that serializes access, or turn on WAL mode with `PRAGMA journal_mode=WAL`.

`restore` trusts the snapshot. A bad row could produce an `Order` whose state violates aggregate invariants. The current `restore` builds the record directly without re-checking. That's defensible when the snapshot came from a valid order originally, and risky when a hand-edited row or migration bug slips through. A paranoid `restore` would re-run invariants, at the cost of going through the public API (re-emitting events) or duplicating validation logic. The project picks one tradeoff and records the choice.

Takeaway

Kata 6 made the repository pattern plausible because an in-memory adapter worked. This kata makes it load-bearing: SQLite slots in behind the same record of two functions, and the use case, handler, and scenario tests run against either backend unchanged. The domain code never learned what storage looks like because the type system gave it no way to find out, and systems built on that constraint survive a storage migration five years later.

The snapshot/restore pair is the right Gleam answer to the serialization boundary. Other languages reach for `@JsonSerializable` annotations or reflection-based ORMs; Gleam asks you to write the encoder by hand, so when the schema drifts the function to update is the one whose field list no longer matches the type.

What's next

The stack is now production-shape: typed domain, events as facts, repositories as interfaces, bounded contexts, HTTP boundary, composition root, and disk persistence. Everything beyond is specialization.

10. Kata 9: SQLite Repository

The final chapter steps out of katas and into practice: what experience teaches that the exercises don't, what to skip, when to escalate, and how to keep the codebase from drifting back into the soup the patterns exist to prevent.

11. Kata 10: Wiring and Configuration

Kata 9 built and tested the SQLite adapter in isolation. Time to hook it up.

`gleamlang_katas.gleam` still says:

```
let assert Ok(repo) = order_repo.in_memory()
```

Two `OrderRepos` exist now, one in an actor and one over sqlite, so the implementation swaps without touching anything downstream.

To test the SQLite wiring, save an order, restart the server, and `GET` the order back. The `OrderRepo` choice also becomes configurable from the command line.

Either via:

```
ORDER_REPO=in_memory
```

or

```
ORDER_REPO=sqlite  
ORDER_DB=my_orders.db
```

New Tricks

envoy handles env var access

```
import envoy  
  
envoy.get("MY_ENV_VAR") // -> Result(String, Nil)
```

`envoy.get` returns `Ok(String)` when the env var has a value, and `Error(Nil)` when it doesn't.

Environment variables always arrive as strings.

Result-handling composes with it the usual way:

11. Kata 10: Wiring and Configuration

```
envoy.get("PORT")
|> result.try(int.parse) // it should become an int
|> result.unwrap(8080) // default value
```

Organizing the Config

The wrong shape:

```
pub type Bad_Config {
  Bad_Config(repo_kind: String, db_path: String)
  // ~~~~~ stringly typed; needs validation everywhere it's used
}
```

The right shape:

```
pub type RepoBackend {
  InMemory
  Sqlite(path: String)
}

pub type Config {
  Config(repo: RepoBackend, port: Int)
}
```

`Bad_Config` is harder to use, since the type system cannot help you set it up correctly. You'd have to scan the code for valid values of `repo_kind`.

Reading the `Config` definition explains what it can be, and hints at how callers will use it. The `db_path` only exists when the variant is `Sqlite`, so you can't accidentally read it when the backend is `InMemory`.

This is the cleanest example of “make illegal states unrepresentable” in the whole book. `Bad_Config` lets (`"in_memory"`, `"/some/path.db"`) exist as a value even though the path is meaningless without `sqlite`; the typed `RepoBackend` makes that pairing un-constructable. The slogan from Kata 1 applied to configuration.

The shape mirrors HTTP input parsing. Untyped strings arrive, a parser turns them into typed values, and the typed values flow through the system.

Note: Extending this to connect to postgres adds the connection uri, or the `host_name` and db name, as fields of a `Postgres` variant of `RepoBackend`. Something like:

```
pub type ExtendedRepoBackend {
  InMemory
  Sqlite(path: String)
  PostgresURI(uri: String)
  PostgresParts(host_name: String, db_name: String, ...)
}
```

Config -> Repo

A config value passes into a function that produces a running adapter.

```
fn build_repo(backend: RepoBackend) -> Result(OrderRepo, String) {
  case backend {
    InMemory ->
      order_repo.in_memory()
      |> result.map_error(string.inspect)

    Sqlite(path) -> {
      // opening a connection can fail:
      use conn <- result.try(
        sqlight.open(path) |> result.map_error(string.inspect)
      )
      order_repo_sqlite.sqlite(conn) |> result.map_error(string.inspect)
    }
  }
}
```

Error types collapse to `String` here. `actor.StartError` and `sqlight.Error` don't share an ancestor, so the composition root flattens them. `string.inspect` works at this layer, since the only consumer of the error is a human (or agent) reading the panic message.

11. Kata 10: Wiring and Configuration

Task

1. Add the dependency

```
gleam add envoy
```

2. Add RepoBackend and Config types

In `gleamlang_katas.gleam` read the config, and turn it into an `OrderRepo`.

3. Implement `load_config`

Env vars

Read `ORDER_REPO` and `ORDER_DB`, and `PORT` into typed values.

Order Repo

Default to `InMemory` when `ORDER_REPO` is unset. Crash if the value is set and invalid.

`ORDER_DB` defaults to `"orders.db"`.

Port

Default Port: 8080. An unparsable value should crash.

4. Implement `build_repo`

The factory given above, but try doing it without looking.

Its signature is `fn(RepoBackend) -> Result(OrderRepo, String)`

5. Update main

```
pub fn main() -> Nil {
  let config = load_config()
  let assert Ok(repo) = build_repo(config.repo)
  let deps = router.Deps(order_repo: repo)
  let handle = router.handle(deps, _)
```

```

let secret = wisp.random_string(64)
let assert Ok(_) =
  wisp_mist.handler(handle, secret)
  |> mist.new
  |> mist.port(config.port)
  |> mist.start
process.sleep_forever()
}

```

6. Update .gitignore

```

orders.db
orders.db*

```

Don't commit the database. The glob also covers SQLite's sidecar files.

7. Verify persistence

The Hurl scripts from chapter 09 (`dev/hurl/`) do the round-trip without any per-step `kill/restart` choreography:

```

# 1. start with SQLite, create + read one order
ORDER_REPO=sqlite gleam run &
hurl --test --variables-file dev/hurl/vars.env dev/hurl/create.hurl
hurl --test --variables-file dev/hurl/vars.env dev/hurl/get.hurl
kill %1

# 2. restart, prove the order survived
ORDER_REPO=sqlite gleam run &
hurl --test --variables-file dev/hurl/vars.env dev/hurl/get.hurl
kill %1

```

If the second `get.hurl` passes, the wiring is done. A 404 means something in the save path isn't writing to disk.

Without Hurl, the same shape with curl:

```

ORDER_REPO=sqlite gleam run &
curl -X POST "http://localhost:8080/orders?order_id=ORDER-001&customer_id=CUST-1"
curl http://localhost:8080/orders/ORDER-001

```

11. Kata 10: Wiring and Configuration

```
kill %1
```

Inspect the SQLite file directly to confirm bytes landed:

```
sqlite3 orders.db 'SELECT id, length(data) FROM orders;'
```

Hints: what to do

1. Read env vars at the top of `main`, once. `envoy.get` inside a handler or use case means configuration has leaked downward; pull it back to `load_config` and pass typed values through `Deps`.
2. Defaults live in `load_config`, not in `build_repo`. By the time `build_repo` runs, the backend choice is decided. Splitting parsing from construction keeps each function small.
3. The factory returns `Result`, not panics. `let assert Ok(...)` is the caller's choice. `build_repo` returns a value so tests can exercise it without crashing the test runner.
4. Open the SQLite connection once at startup, hand it to the repo, and don't close it. The OS closes it on process exit. (Tests open `:memory:` per test, as kata 9 established.)
5. Pick a case convention for `ORDER_REPO` values (lowercase reads as more shell-friendly) and reject unrecognized values rather than guessing. The Critique section returns to this.

Walk-through

`load_config`:

```
import envoy
import gleam/int
import gleam/result

fn load_config() -> Config {
  let repo = case envoy.get("ORDER_REPO") {
    Ok("sqlite") -> {
      let path = envoy.get("ORDER_DB") |> result.unwrap("orders.db")
      Sqlite(path)
```

```

    }
    Ok("memory") -> InMemory
    Error(nil) -> InMemory      // unset: default
    Ok(_) -> InMemory           // unrecognized: see critique
  }

  let port =
    envoy.get("PORT")
    |> result.try(int.parse)
    |> result.unwrap(8080)

  Config(repo: repo, port: port)
}

```

The function is total, since any environment produces a `Config`, which keeps `main` straight-line.

`build_repo` is the factory from “New Gleam fundamentals.” It’s the one place that turns a typed `RepoBackend` into a running `OrderRepo`, with all error paths funneled to `Result(_, String)`.

main becomes choreography:

```

let config = load_config()           // env → typed
let assert Ok(repo) = build_repo(config.repo) // typed → adapter
let deps = router.Deps(order_repo: repo) // adapter → bundle
let handle = router.handle(deps, _)   // bundle → handler
// ... start Mist on config.port ...

```

The boundary between “parse the world” and “do work” sits in the first three lines, and the `let assert Ok` on line 2 is the only place this file panics on adapter construction.

Critique

Env vars suit one-process apps and stop scaling past dev/staging/prod with secret rotation. The source then becomes a config file or secrets manager, but the shape of `load_config` doesn’t change: `envoy.get` becomes `secrets_manager.fetch`, and the typed `Config` flows through unchanged. The typed-config boundary pays back here.

File paths aren’t validated up front. `Sqlite("/no/such/dir/orders.db")` doesn’t fail until `sqlight.open` runs, which is the right moment for the error to arrive; pre-validation would duplicate the open logic.

11. Kata 10: Wiring and Configuration

Falling back to `InMemory` on an unrecognized `ORDER_REPO` is the worst bug in the kata. A typo (`ORDER_REPO=sqlight`) yields a fresh in-memory repo on every restart, silently. Production code should log a warning or refuse to start on any non-empty unrecognized value, so that “unset” and “set to garbage” land on different code paths.

Migrations and concurrency are out of scope. `CREATE TABLE IF NOT EXISTS` works for v1; the day a `version INT` column joins, a migration tool becomes necessary. SQLite serializes writes through one connection, which suits one Mist process; heavier load wants WAL mode or an actor that serializes connection access. Both are mechanical additions when they matter.

The tests don’t exercise the factory end to end. `order_repo_sqlite_test` constructs the repo directly, which is right for unit tests; an integration test that sets env vars and asserts the right `Config` and `OrderRepo` come out would prove the wiring as the config surface grows.

Takeaway

Configuration is a boundary like HTTP. Untyped strings arrive at the edge, a parser turns them into typed values, and everything downstream works with the typed form. The HTTP handler does this for request bodies; `load_config` does it for env vars.

Keeping the untyped world at the edges earns its keep over time more than the domain model itself or the repository pattern. A `Dynamic` that leaks past `main.gleam` is a future bug; a sum type meeting a stringly-typed flag downstream is a future refactor. The in-memory repo from kata 6 was never only a teaching tool, since it runs the test suite and local development. The SQLite repo slots in behind the same interface, and the layers above can’t tell which is which. The composition root is the one file that knows.

What’s next

The next and final chapter, *Putting It Into Practice*, distills the experience of shipping software with these patterns: what to reach for on Monday morning, what to skip, when to escalate from one pattern to the next, and how to discuss any of it with colleagues who haven’t read the book. The katas teach the patterns; the closing chapter covers application.

12. Putting It Into Practice

The katas teach the patterns. The harder skill is reading a working codebase and judging which of these tools earns its weight today and which doesn't, yet or ever. This chapter is opinionated; disagreement is welcome where context differs.

Don't start with all of it

If you're starting a fresh Gleam project tomorrow, do *not* start with:

- A `domain/`, `application/`, `infrastructure/` folder split
- An `OrderRepo` interface before there's a second implementation
- An event bus before something needs to react to a transition
- A composition root with a `Deps` record before there are multiple deps
- Bounded contexts before there are two distinct sub-models

Start with:

- One module per concept (`order.gleam`, `customer.gleam`)
- Smart constructors that return `Result`
- Functions that take and return values
- A flat `src/` directory until you feel pressure to split

That foundation covers most of what most apps need.

The patterns in this book are answers to specific problems. Apply them once you've hit the problem. Introducing them preemptively is how DDD earns its reputation for ceremony.

Minimum viable DDD

For a small project (under ~10k LoC, one team), here's what earns its weight:

Pattern	Worth it for...
Smart constructors + Result errors	every project, day one
Opaque types for domain values	every project, day one
Aggregate root + state-transition methods	when 3 mutations cluster around the same fields
Repository as record of functions	when you have 2 storage backends OR want to test without one
Domain events	when something must react without coupling, or you want an audit log
Bounded contexts as folders	when the model honestly has two sub-models, not just two screens
Use cases as named functions	when the app has more than ~5 endpoints
Composition root with Deps bag	when you have 3 deps that travel together
result.try chains in use cases	as soon as use cases call multiple fallible things
Idempotent event handlers	as soon as something replays events (which something will)
The HTTP boundary as pure translation	every HTTP-served project

When you're not sure whether a pattern earns its weight, wait. Adding it later, once the pain is real, costs less than carrying scaffolding through six months of nobody needing it, and removing an entrenched abstraction is politically harder than adding the missing one.

Where bounded contexts come from

Contexts emerge from use. They show up when team boundaries harden, when two parts of the model start disagreeing about what the same word means, or when a one-line type change drags three people into the same PR. Whiteboard contexts drawn on day one are guesses; lived contexts are evidence.

You'll know it's time to split when:

- The **Customer** type has 14 fields and any given function uses 3
- A change to **Customer** blocks a team that doesn't care about the field you're touching
- People start writing **BillingCustomer = Customer with extra fields** workaround types to escape the noise

- A code review for a tiny feature requires sign-off from three teams

That's when you reach for the bounded contexts chapter and split.

The inverse holds, too. Two contexts that always change together aren't two contexts. If `customer_billing.gleam` and `customer_marketing.gleam` show up in the same PR every time, merge them and stop pretending.

Refactor moves as the system grows

These shape-changes show up in roughly this order, each provoked by its own pain.

“My CRUD endpoints are getting hard to test”

Pull the business logic into a domain function and test that directly. The endpoint becomes a 5-line translator, moving from Rails controller to use case.

“I keep re-validating the same string in 8 places”

Make it a value object: an opaque type with a smart constructor that validates once at the boundary. (Kata 1.)

“I have a 12-field record and I add one more every sprint”

Find the sub-clusters hiding inside. Often there's an aggregate root pretending to be a record, with two value objects pretending to be fields. (Katas 1, 2, 4.)

“Every endpoint takes 9 arguments now”

Bundle them into a `Deps` record and pass it down. (Chill-DI bag.)

“I want X to happen after Y but X shouldn't know about Y”

Introduce an event that Y emits and X listens for. (Kata 5.)

“Tests are slow because they hit the database”

Repository as record of functions; pass `in_memory()` in tests. (Kata 6.)

“Two teams are stepping on each other in the same module”

Bounded contexts. (Kata 7.)

“Restarting the server loses everything”

SQLite (or Postgres) adapter, same repo interface. (Kata 9.)

Something concrete in the codebase provokes each move: write the straightforward thing, hit the wall, refactor toward the pattern. The wall is information unavailable on day one and unrecoverable from speculation.

Signs you’ve over-applied DDD

A pattern outlasts its reason and turns into ceremony. The shapes that line tends to take:

- A Repository interface with one implementation, a wrapper rather than a port. If you aren’t swapping implementations between tests and prod, or between backends, the indirection isn’t paying for itself yet.
- Code reviews that argue whether something is an Entity or a Value Object. Productive for five minutes, a red flag at thirty.
- An aggregate root with 47 methods, usually two aggregates trying to be one, or an aggregate that wants to be a domain service.
- Bug fixes that touch six files because of “the layering.” Layers are supposed to contain changes; when small changes ripple wide, the layering is doing the opposite of its job.
- A Saga or Process Manager added “just in case.” These solve specific scaling problems, and without the problem they’re cosplay.
- A Deps record with one field. Just pass the field; reach for the bag once there are three things travelling together.

When you spot one, the fix is usually to delete the abstraction and inline what it was wrapping. Subtractive refactoring is the most underrated tool in this toolkit.

Testing in a DDD-shaped pyramid

The pyramid the katas built:

	few, slow:
end-to-end	wisp/simulate -> handler -> use case
	-> repo -> assert response
use case	some, medium:
with in-memory	place_order.run against in_memory()
adapters	repo. No HTTP. No real DB.
pure domain function	many, fast:
unit tests	order.add_line(...) with various
	inputs. No IO. Milliseconds.

The pyramid only inverts when there's nothing pure to test. The non-DDD shape (logic smeared across HTTP handlers, ORM callbacks, and DB triggers) forces every test to the top, because no clean domain layer exists to exercise on its own. The testing payoff from DDD is that the bottom of the pyramid exists at all.

Property-based testing belongs at the bottom. Smart constructors and state-transition methods are obvious property targets. (“For all non-empty strings, `email.new` either succeeds or returns a typed error.”)

Composition root in production

The `main.gleam` from kata 8 was the minimum. A production composition root does more:

```
pub fn main() {
  // 1. Read configuration from env vars, at the top, exactly once
  let config = config.from_env()

  // 2. Open shared resources (DB pool, HTTP clients, secret keys)
  let assert Ok(conn) = sqlight.open(config.database_path)
  let secret = config.session_secret

  // 3. Construct adapters
```

```
let assert Ok(order_repo) = order_repo_sqlite.sqlite(conn)

// 4. Bundle dependencies
let deps = Deps(
  order_repo: order_repo,
  clock: time.now,
  // ... more as the app grows
)

// 5. Build the request handler from deps
let handle = web.router(deps)

// 6. Start the server (supervised when you need crash recovery)
let assert Ok(_) =
  wisp_mist.handler(handle, secret)
  |> mist.new
  |> mist.port(config.port)
  |> mist.start

process.sleep_forever()
}
```

Configuration enters once, at the top. Scattering `env.get(...)` calls through the codebase always rots. Resources have known lifetimes: the DB connection lives for the process, tests open their own, and “lazy pool that initializes on first use” reliably ships the “works locally, breaks in production” class of bug. Failure to initialize crashes the boot, so a missing DB exits the process before the first request instead of three minutes later when a user notices. Supervision wraps everything when you need it; plain HTTP servers don’t, since Mist supervises itself. Below the composition root, nothing else has `let assert Ok(...)` on adapter construction.

Talking to non-DDD colleagues

Most engineers haven’t read Evans. Saying “the aggregate’s invariant requires that the bounded context emit a domain event so the anti-corruption layer can translate it” will reliably glaze the room.

Translate down:

DDD term	Plain version
Aggregate	“The cluster of stuff that has to be saved together”
Aggregate root	“The thing the rest of the code talks to”
Invariant	“A rule we always want to be true”
Bounded context	“The slice of the system this team owns”
Value object	“A type that’s just its data, like a number”
Entity	“Something with an ID, same person across renames”
Repository	“How we load and save the X”
Domain event	“Something that happened, that other parts care about”
Use case	“The thing one button does”
Smart constructor	“The function that makes a valid one”
Anti-corruption layer	“The translator between us and the other team’s API”
Ports and adapters	“The implementation plugs into an interface”
Ubiquitous language	“We use the same words the business uses”

Reach for jargon when it makes the conversation shorter, plain language when it makes the conversation clearer. The system is what matters; the vocabulary is the tool you use to talk about it.

What's beyond the foundation

The katas covered the load-bearing patterns. Beyond them, larger ideas show up in production systems, roughly in this order:

1. Event sourcing

Store the event log and derive current state by replaying it. The log is the source of truth, and aggregate state is just a projection of it. You get an audit log for free, and “what was this customer’s state on June 14?” becomes a query instead of a forensic exercise.

When to reach: high audit or compliance demand; complex history the team has to reason about; “how did we get into this state?” as a recurring debugging question.

When to skip: most CRUD apps. The complexity overhead lasts forever.

2. CQRS: Command/Query Responsibility Segregation

Separate models for writes (commands) and reads (queries). Aggregates handle the writes while denormalized projections handle the reads.

When to reach: read-heavy apps where the aggregate shape is the wrong shape to query; cross-aggregate reports; materialized views.

When to skip: writes and reads use roughly the same shape.

3. Sagas / Process Managers

Coordinated multi-step workflows that span aggregates or services: “place order, reserve inventory, charge card, ship, email receipt.” Each step is a use case, and the saga handles sequencing, retry, and compensation when a step fails.

When to reach: multi-aggregate workflows that cross service boundaries; explicit compensation logic that the business cares about.

When to skip: workflows that fit in one transaction.

4. Distributed event delivery

Moving from in-process function calls to a message broker (Kafka, NATS, RabbitMQ) or BEAM distribution. Aggregate code stays identical; only the wiring at the composition root changes, publishing to the broker instead of calling handler functions directly.

When to reach: services in different processes or data centers; at-least-once delivery, retries, dead-letter queues; consumers that should be independently restartable.

When to skip: monolith, or any case where a synchronous in-process call is fine.

5. Anti-corruption layers in earnest

Adapters between your domain and a system whose model you don’t control. The ACL translates their vocabulary into yours, so the rest of your code only ever sees your types.

When to reach: integrating with a system you don’t own and can’t change; multiple teams whose APIs evolve on their own schedule.

When to skip: small project, one team, all code under your control.

Each of these is its own book. The kata progression is the foundation that makes those books readable.

What the kata omits, and what you'd add to ship it

Two load-bearing pieces sit just outside the kata's scope, and they're the first things any extension runs into.

Customer endpoints

The HTTP boundary accepts `customer_id` as a string and never checks whether the customer exists. There are no `POST /customers` or `GET /customers/:id` endpoints, even though `src/customer.gleam` defines the type. If you add them, you have to decide how Ordering should treat the cross-context reference:

- (a) Pre-validation: call `CustomerRepo.find` before accepting an order; reject unknown ids.
- (b) Async subscription: Customer publishes `CustomerCreated`; Ordering keeps a local index.
- (c) Trust by default: accept any id, reconcile via a periodic job if needed.

Pick one deliberately when it matters. The kata implements (c) by omission, which is fine for a learning project but worth being honest about for anything that ships.

JSON request bodies

The HTTP boundary kata uses path parameters only. Real APIs take JSON bodies, which means a decoder per endpoint that translates `Result(InputStruct, json.DecodeError)` into either a 422 with details or a use-case call. The shape doesn't change (it's another translation layer in the shell), but it's its own skill, covered in `Wisp + gleam_json` cookbook material.

A few patterns the book skipped on purpose

Worth flagging because you'll run into them in other books and wonder where they went:

- Domain Services: operations that don't naturally belong on any one entity. ("Transfer money from A to B", since neither account "owns" the transfer.) In Gleam, that's a top-level function that takes both. The "Service" object is OO baggage.

12. Putting It Into Practice

- Specifications: composable query objects like “active customers in California with last order over \$100.” Useful in OO languages where queries have to be objects. In Gleam, write functions and compose them with `|>`.
 - Factories as classes. Gleam’s smart constructors already are the factory, so no additional ceremony earns its keep.
 - Layered architecture as packages: `application/`, `domain/`, `infrastructure/`. Often right in spirit and often expensive in practice. Start flat and let coupling pressure suggest the split.
-

A few things that aren’t DDD but pair well with it

- Property-based testing for smart constructors and pure domain functions. `gleam_qcheck` or similar.
 - Snapshot testing for stable JSON outputs like HTTP responses and events. Catches accidental shape changes you’d otherwise notice in production.
 - Schema migrations as ordered SQL files (`dbmate`, `sqitch`, `gorrion`). Plain SQL keeps you honest about what the database does.
 - OpenTelemetry traces through the use case $\rightarrow$ handler $\rightarrow$ adapter call graph. The DDD layering makes the trace structure read like the code.
 - Static dependency-graph linting, so you find out the moment Ordering accidentally imports something from Shipping.
-

A short closing

The vocabulary, the patterns in idiomatic Gleam, and some judgment about when to wait are now in hand, along with the moves that keep each pattern small: records of functions in place of interface hierarchies, smart constructors in place of Factory classes, closures in place of DI containers, modules in place of multi-package layouts.

Muscle memory is what a book can’t give. Build something small with these tools and ship it. Most of what the project needs was already in the first four chapters; the rest sits in the back of the mind, available when a problem justifies pulling it out.

use in Gleam — an in-depth explainer

Gleam’s **use** is one keyword with one mechanical rule, but it shows up in several different shapes. Once you internalize the rule, every shape is just an application of it. This doc walks through the rule, then the patterns you’ll hit most often.

1. The mechanical rule

```
use <bindings> <- f(arg1, arg2, ..., argN)
<rest of block>
```

desugars to exactly:

```
f(arg1, arg2, ..., argN, fn(<bindings>) { <rest of block> })
```

That’s the whole feature. **use** takes the rest of the block, wraps it in an anonymous function, and passes that function as the **last argument** to **f**.

A few consequences fall out of that one rule:

- The helper **f** is just a normal function that happens to take a callback as its last parameter. Nothing magical about it.
- Whatever the callback returns is what **f** returns (in the success branch). So your block’s last expression is the function’s value.
- **use** must be followed by something — there has to be a “rest of the block” to lift into the callback. A **use** line on its own is an error.

Why Gleam has this

Gleam is **expression-based and has no early return**. Without **use**, every “check then continue” pattern becomes a nested **case**, and chains of fallible operations become a pyramid.

Concretely: `Customer.new` in this repo is strict — it takes `(id: CustomerId, name: String, email: Email)`, all pre-validated. But somewhere at the boundary (HTTP handler, CLI, DB row) you have raw strings that need to be turned into those value objects first. That “raw → validated” factory is where the chain lives. Without `use` it looks like this:

```
case email.new(raw_email) {
  Error(e) -> Error(InvalidEmail(e))
  Ok(email) -> case customer.new_id(raw_id) {
    Error(e) -> Error(e)
    Ok(id) -> customer.new(id, raw_name, email)
  }
}
```

With `use` and `result.try`, that pyramid flattens into a sequence:

```
import gleam/result

use email <- result.try(email.new(raw_email) |> result.map_error(InvalidEmail))
use id <- result.try(customer.new_id(raw_id))
customer.new(id, raw_name, email)
```

Each line is one validation step. There is no nesting, no manually re-propagating errors, no `Ok(...)` → `case ...` ladder. If any step returns `Error`, the whole block short-circuits with that error; if all succeed, the final expression is the function’s return value.

(Adding this boundary factory would require adding `InvalidEmail(EmailError)` to `CustomerError` so the email error has a place to live — see section 4.)

`use` is Gleam’s answer to Rust’s `?`, Haskell’s `do` notation, and Clojure’s macros — but with one syntactic rule covering all the cases instead of a separate construct per case.

2. The shape of the helper

Any function with this general signature is a candidate for `use`:

```
fn helper(arg1, arg2, ..., callback: fn(...) -> R) -> R
```

The helper decides three things:

1. **What arguments the callback receives** (zero or more, of any types).

3. Pattern — guard / short-circuit (zero-arg callback)

2. **When the callback runs** (always? only on success? only on failure?).
3. **What to return when the callback doesn't run** (the short-circuit value).

That's the design space. Every pattern below is a different choice across those three knobs.

3. Pattern — guard / short-circuit (zero-arg callback)

The helper takes a precondition. If it passes, it runs the callback (the rest of your block). If it fails, it returns an error directly.

```
fn require_same_currency(  
  a: Money,  
  b: Money,  
  then: fn() -> Result(t, MoneyError),  
) -> Result(t, MoneyError) {  
  case a.currency == b.currency {  
    False -> Error(CurrencyMismatch)  
    True -> then()  
  }  
}  
  
pub fn add(a: Money, b: Money) -> Result(Money, MoneyError) {  
  use <- require_same_currency(a, b)  
  new(a.amount + b.amount, a.currency)  
}
```

Key signals it's a guard:

- The callback's parameter list is empty (`fn() -> ...`).
- At the call site you write `use <- f(...)` with no binding.
- The check is the entire contract of the helper.

Other guard examples worth writing:

```
fn require_authenticated(session, then: fn() -> Result(a, e)) -> Result(a, e)  
fn require_role(session, role, then: fn() -> Result(a, e)) -> Result(a, e)  
fn require_non_empty(s: String, then: fn() -> Result(a, FieldError)) -> Result(a, FieldError)
```

You're effectively building little control-flow combinators for your domain.

4. Pattern — Result chaining (one-arg callback)

The helper *unwraps* a `Result`, passes the inner value to the callback, and short-circuits on error. This is `result.try`:

```
// from gleam/result
pub fn try(
  result: Result(a, e),
  apply: fn(a) -> Result(b, e),
) -> Result(b, e)
```

Used with *use*. The natural home for this in our codebase is a `from_raw` boundary factory that pairs with the strict inner `Customer.new`:

```
// Strict inner constructor (already in customer.gleam) - no `use` needed,
// because every argument is already a validated value object.
pub fn new(
  id: CustomerId,
  name: String,
  email: Email,
) -> Result(Customer, CustomerError) {
  case string.length(name) {
    0 -> Error(EmptyName)
    _ -> Ok(Customer(id, name, email))
  }
}

// Boundary factory - turns raw strings into a Customer, composing errors.
// This is where the `use` chain belongs. Requires CustomerError to gain
// an `InvalidEmail(EmailError)` variant.
pub fn from_raw(
  raw_id: String,
  raw_name: String,
  raw_email: String,
) -> Result(Customer, CustomerError) {
  use email <- result.try(email.new(raw_email) |> result.map_error(InvalidEmail))
  use id <- result.try(new_id(raw_id))
```

```

    new(id, raw_name, email)
}

```

Two layers, two responsibilities:

- `new` enforces *Customer-level* invariants and trusts its value-object inputs. No `use` because there's nothing to short-circuit on.
- `from_raw` is the boundary — turns untrusted strings into trusted values, composes the errors, then delegates to `new`.

This is the moral equivalent of:

- Rust: `let email = email::new(raw_email).map_err(InvalidEmail)?;`
- Haskell: `email <- liftError InvalidEmail (Email.new raw_email)`
- F#: `let! email = ...`

Key gotcha — error types must match. `result.try` requires the input's error type to match the function's return error type. When they differ, lift the inner error with `result.map_error(WrappingConstructor)` — exactly what the email line above does. This is why composing modules pushes you toward each module owning its own error type and *wrapping* others rather than sharing one giant error enum.

5. Pattern — Option chaining

Same shape as Result chaining, with `option.then`:

```

import gleam/option.{type Option, Some, None}

pub fn shipping_eta(customer: Customer) -> Option(Date) {
  use address <- option.then(customer.address)
  use zone <- option.then(zone_for(address))
  Some(eta_for_zone(zone))
}

```

Returns `None` if any link in the chain is `None`, otherwise `Some(...)`.

6. Pattern — resource acquisition (with-style)

A helper that **always** runs the callback but wraps it in setup/teardown. Same idea as **with-open** in Clojure or **with** in Python.

```
fn with_db_connection(  
  config: Config,  
  body: fn(Connection) -> a,  
) -> a {  
  let conn = connect(config)  
  let result = body(conn)  
  close(conn)  
  result  
}  
  
pub fn fetch_user(config: Config, id: Int) -> Result(User, DbError) {  
  use conn <- with_db_connection(config)  
  query(conn, "select ...", [id])  
}
```

The connection always closes, even if `body` returned an error, because `close(conn)` runs after `body(conn)` returns. (For real cleanup you need to handle exceptions; Gleam has libraries for this — the pattern is what matters here.)

7. Pattern — list iteration (less common)

You can use `use` with `list.try_each` and friends to iterate fallibly while keeping the body flat:

```
use _ <- list.try_each(orders) // returns Error on first failure  
  process_order(_)
```

Honestly, for most list work, `list.map` and `list.try_map` in pipelines read better than `use`. Reach for `use` when the body is multi-line or has its own internal `use` chain.

8. Pattern — custom DSLs

Once you see `use` as “callback with this last-arg shape,” you can build your own combinators:

```
fn assert_field(name: String, value: String, then: fn(String) -> Result(a, FieldError))
  -> Result(a, FieldError) {
  case string.trim(value) {
    "" -> Error(FieldError(name, "must not be empty"))
    trimmed -> then(trimmed)
  }
}

pub fn parse_form(raw: Form) -> Result(ValidForm, FieldError) {
  use name <- assert_field("name", raw.name)
  use email <- assert_field("email", raw.email)
  use age_str <- assert_field("age", raw.age)
  use age <- result.try(parse_int(age_str))
  Ok(ValidForm(name, email, age))
}
```

Each `use` line is a single concern; the body is a flat list of validations. Without `use` this would nest five levels deep.

9. Syntactic variants

```
use <- f(args)           // zero-arg callback   (guards)
use x <- f(args)          // one-arg callback    (try, then)
use x, y <- f(args)       // multi-arg callback (rare; e.g. fold-style helpers)
use _ <- f(args)          // discard the binding (run for side effect)
```

The number of bindings on the left of `<-` must match the callback’s parameter count exactly. Compiler enforces this.

10. Type constraints (the part that bites)

The callback's return type has to line up with what the helper expects. The two most common mismatches:

Mismatched error types in `result.try`:

```
// won't compile - EmailError   CustomerError
use email <- result.try(email.new(raw))

// fixed - lift the inner error
use email <- result.try(email.new(raw) |> result.map_error(InvalidEmail))
```

Wrong return type from the body:

The body of a `use` block must return what the helper's callback declared. For `result.try`, that's `Result(_, e)`. So the last line of a `use`-chained function is almost always `Ok(something)` or `Error(something)`, not a bare value:

```
use email <- result.try(...)
Ok(Customer(email))    //
Customer(email)        // - type error, expected Result, got Customer
```

11. When NOT to reach for `use`

- **One-shot pattern matching** with no chain: `let assert Ok(x) = ...` is fine (when crashing on `Error` is acceptable) and reads more directly than `use x <- result.try(...)` followed by a single line.
- **The body has only one statement.** Nesting one level isn't a sin. `use` pays off when there are two or more steps to flatten.
- **The helper isn't really doing control flow.** If you find yourself writing a “helper” that just calls the callback unconditionally, the `use` is just obfuscation — drop it.

12. Mental model summary

Forget the syntax for a second. The shape `use ... <- f(...)` says:

“I’m about to do the rest of this function as a callback. `f`, you decide what to do with it: run it, skip it, run it conditionally, run it with cleanup, run it with these arguments. Whatever you return is what I return.”

That’s it. Every pattern in this doc is a different choice the helper makes about how to use that callback. Once helpers exist for guards (your domain), `Result` (`result.try`), `Option` (`option.then`), and resources, most Gleam business logic flattens into a sequence of `use` lines reading top-to-bottom like a recipe.

Compared to other languages:

Need	Gleam	Rust	Clojure	Haskell
Result short-circuit	<code>use x <- try(...)</code>	?	<code>some-></code> / <code>mlet</code>	<code>do + <-</code>
Guard / early return	<code>use <- guard(...)</code>	<code>if cond { return }</code>	macro / <code>when-let</code>	<code>when</code>
Resource handling	<code>use r <- with(...)</code>	Drop trait	<code>with-open</code>	<code>bracket</code>
New control flow	helper + <code>use</code>	macro / <code>fn</code>	macro	monad / <code>fn</code>

Gleam is unusual in answering all four with the same one-syntax-rule mechanism. That uniformity is the whole point.

